DuplexNVC Core Baseline: Full-Duplex MLVC on an Apple M4 Pro

Phase 2 Report: The Core Baseline and Its Extension

SHAMS AL-ANI, American University of Sharjah, UAE
WARDA UL HASAN, American University of Sharjah, UAE
MOHAMMED ALAWADHI, American University of Sharjah, UAE

DuplexNVC targets two-way (full-duplex) video calls coded with a learned video codec on one laptop chip, where the encoder of the local camera and the decoder of the remote participant must share the same processors. Before any improvement can be claimed, a fixed baseline is needed. This report documents that baseline as the Phase 1 feedback defined it: the full MLVC model at 360p, with both networks on the Apple Neural Engine (NE), running a call at 30 frames per second (fps). We describe the two MLVC variants we used, the C++ measurement program we wrote on top of the `libmlvc` library, the tools and conditions under which every number was recorded, the smoke tests and the problems they exposed, a re-measurement of the Phase 1 MLVC-S results, and the full baseline evaluation of speed, concurrency, real-time behaviour, energy, decoding correctness and compression quality. On an Apple M4 Pro the baseline B^* holds 30 fps in both directions with no late frame, and 99% of frames finish within 29 ms of the 33.3 ms deadline. Running the encoder and decoder at the same time on the NE is only 1.11 times faster than running them one after the other, against an ideal of about 2, because each network runs about 1.8 times slower while the other is active. The NE draws 97% of the 65 mJ spent per pair of frames, while the GPU stays idle. Full MLVC needs 31% fewer bits than hardware HEVC for the same luma quality, yet once the picture is shown at 1080p the low 360p resolution, rather than the coding, causes most of the loss. These results point DuplexNVC at contention on the NE and at resolution. Following the feedback on the first version of this report, we then extended the baseline in a second measurement campaign: we traced on which processor every operation of the networks runs, timed each network alone on the GPU and the CPU, converted full MLVC and MLVC-S for 540p, 720p and 1080p, and ran a single 22-hour experiment that measures every combination of model, resolution and processor placement. On the GPU each network is 2.2–2.3 times slower than on the NE and draws several times more power, and every operation of every network runs on the processor it was given. Of 38 combinations, 23 meet 30 fps, 9 only 15 fps and 6 neither. Both networks on the NE is the cheapest placement wherever it keeps up, but it is the first to miss deadlines as the load grows: full MLVC at 540p and MLVC-S at 720p meet 30 fps only when one network moves to the GPU, at about 3.5 times the energy, and full MLVC at 720p reaches only 15 fps in any placement. In total the work in this report took more than 30 hours of laptop time, most of it in a fixed "quiet mode" (AC power, Wi-Fi and Bluetooth off, maximum brightness). This report is a baseline only: the remaining steps of the plan, each of which takes several hours in quiet mode, are described as the plan on which DuplexNVC will build.

CCS Concepts: • **Computer systems organization** → **Heterogeneous (hybrid) systems**; • **Computing methodologies** → *Image compression*; • **Hardware** → *Power estimation and optimization.*

Additional Key Words and Phrases: learned video compression, MLVC, full-duplex video calls, Neural Engine, heterogeneous computing, energy measurement, baseline evaluation

1 Introduction

In a video call each laptop does two jobs at once: it encodes its own camera and decodes the video of the other person, and it must finish both within 33.3 ms for every frame at 30 fps. Learned video codecs of the DCVC line now reach real-time speed [3, 4], and MLVC runs one on a laptop's neural

Authors' Contact Information: Shams Al-Ani, g00113665@aus.edu, American University of Sharjah, Department of Computer Science and Engineering, Sharjah, UAE; Warda Ul Hasan, g00090299@aus.edu, American University of Sharjah, Department of Computer Science and Engineering, Sharjah, UAE; Mohammed Alawadhi, b00108492@aus.edu, American University of Sharjah, Department of Computer Science and Engineering, Sharjah, UAE.

processing unit through a C++ library, `libmlvc`, built on Apple's Core ML framework [6]. Our survey found, however, that every one of these speeds was measured for the encoder or the decoder alone, on one processor, and that none of the learned-codec papers reports energy [1]. A real call needs both directions on the same chip at the same time, and no study has measured what that costs.

DuplexNVC is our proposal for running both directions of a call across the CPU, GPU and Neural Engine of an Apple Silicon laptop and for choosing MLVC's settings from the content of the frame and the energy state of the device. The Phase 1 feedback asked us first to define and measure a *core baseline*: a fixed, simple configuration against which every later DuplexNVC change will be compared, so that any claimed gain is a gain over something concrete and repeatable. The feedback fixed that baseline as full MLVC at 360p with both networks on the Neural Engine.

In Phase 2 we built everything needed to measure it. We wrote a small C++ program on top of `libmlvc` that times the encoder and decoder alone, one after the other, and at the same time, both as fast as possible and paced at 30 fps, and that records every frame. We first ran short smoke tests to make sure the pipeline worked, and fixed the problems they revealed. We then re-measured the MLVC-S configuration of Phase 1 with the new program, which both checked the program and turned the two estimates of Phase 1 into measurements. Finally, we ran the full evaluation of the baseline: the speed of each network alone, the benefit of sharing the Neural Engine, behaviour under a real 30 fps deadline, energy, decoding correctness on every processor, and compression quality at the coded and at the display resolution.

The main result is that the baseline meets real time, but with little margin, and that the Neural Engine does not run the two networks in parallel to any useful degree. Running them at the same time saves only 11% of the time that running them one after the other takes, while the GPU sits idle and the Neural Engine draws almost all the energy. The second result is that compression is strong, but that at the size a viewer actually sees, the 360p resolution limits quality more than the coding does. Together these results identify where DuplexNVC can gain: moving work off the Neural Engine, scheduling the two directions so that they compete less, and spending the time gained on resolution.

The feedback on the first version of this report asked for more than one fixed point: it asked how the baseline behaves on the other processors of the chip, at the other resolutions and model sizes of our survey, and with every assignment of the encoder and decoder to the NE, the GPU and the CPU. We therefore extended the baseline in a second campaign (Section 7). We first traced the placement of every operation and timed each network alone on the GPU and the CPU, then converted the two MLVC variants for 540p, 720p and 1080p and prepared their input streams, and finally ran one experiment of about 22 hours that measures every combination of model, resolution and placement, both as fast as possible and paced at 30 fps, with energy. Because each measurement must be taken with the laptop in a fixed quiet state (Section 3.3), with no other work on it, these experiments take many hours: the work in this report amounts to more than 30 hours of laptop time (Table 8).

We stress that this report presents a *baseline*, not DuplexNVC itself. The 22-hour experiment finished shortly before the deadline of this report; its results are included, but their quality at every setting is still to be measured. The remaining steps of the plan (quality at every resolution, correctness of every encoder and decoder pairing, energy on battery, the cost of scene analysis, the multi-person data and an on-device HEVC anchor) each require several more hours with the laptop in quiet mode and could not be completed by the deadline; they are given in Section 9 as the plan on which DuplexNVC will build.

The rest of the report is organised as follows. Section 2 describes the two MLVC variants and defines every metric. Section 3 describes the baseline, the platform, the software, the measurement conditions, the measurement program and the data. Section 4 reports the smoke tests and Section 5

the re-measurement of Phase 1. Section 6 reports the baseline results and Section 7 its extension to other processors, resolutions and model sizes. Section 8 discusses the results and their limitations, and Section 9 gives the plan for the next phase. The baseline was measured on 10 November 2026 and the extension on 12 and 13 November 2026, all on the same device. The code, scripts and raw records are available in our GitHub repository (Section 3.9).

2 Background

2.1 MLVC and MLVC-S

MLVC is a conditional learned video codec of the DCVC family [3, 6]. For every frame, an encoder network turns the frame into a compact latent representation, which is quantised and entropy-coded into bits; a decoder network rebuilds the frame from these bits. Both networks are conditioned on features carried over from the previously decoded frame, so the decoder must see every frame in order, and an intra-coded frame (I-frame) is inserted periodically to reset this carried state. A single quality index, the quantisation parameter (QP), trades bitrate against quality: a higher QP gives fewer bits and lower quality. `libmlvc` wraps the two networks with pre-processing (colour conversion and padding), post-processing, entropy coding and rate control, and runs the networks through Core ML on a processor chosen by the caller: the Neural Engine (NE), the GPU or the CPU.

We used two variants (Table 1). The *full* MLVC model (`dmc61sbr_reglu`) is the larger network. We used its public weights trained for PSNR (`psnr_v1`) and converted them ourselves into a Core ML model at half precision (fp16) for a 640×368 input, which is the 360p frame padded to a multiple of 16 pixels. *MLVC-S* (`dmc61sbr_mini_reglu`) is a smaller network trained for perceptual quality (`perceptual_v1`); it ships with `libmlvc` as a ready 12.8 MB bundle that already contains models for four input sizes, from 320×192 to 960×544. Phase 1 used MLVC-S at 540p and QP 22. Following the feedback, the Phase 2 baseline uses the full model with PSNR weights at 360p and QP 26: with PSNR weights its rate–distortion results can be compared directly with classical codecs, which are also reported in PSNR, and 360p is the size at which the MLVC paper reports its laptop speeds [6], so our speeds can be checked against it. In this report MLVC-S is used only to check the new measurement program against Phase 1.

Table 1. The two MLVC variants used in this report.

	Full MLVC (baseline)	**MLVC-S** (Phase 1)
Model	`dmc61sbr_reglu`	`dmc61sbr_mini_reglu`
Weights	`psnr_v1`, trained for PSNR	`perceptual_v1`, trained for perceptual quality
Core ML model	Converted by us with `mlvc-tools`, fp16	Bundle shipped with `libmlvc` (12.8 MB)
Model input sizes	640×368	320×192, 432×240, 640×368, 960×544
Used at	640×360 (360p), QP 26	960×540 (540p), QP 22
Role here	Baseline B^* (E1–E7)	Check of Phase 1 (E3a)

2.2 Taking Turns and Sharing the Neural Engine

A *frame pair* is one frame encoded for the outgoing stream and one frame decoded from the incoming stream; a call at 30 fps must serve one frame pair every 33.3 ms. There are two simple ways to serve it on one processor. In *taking turns*, a single thread encodes the local frame and then decodes the remote frame, strictly one after the other. In *sharing*, an encoder thread and a decoder thread run independently and submit their work to the Neural Engine at the same time, leaving it

to the system to interleave the two. Taking turns is the simplest correct program; sharing is what a straightforward duplex application does. The question is how much faster sharing is, which tells us whether the Neural Engine can run the two networks in parallel.

Let $c_{x,k}$ be the measured time of the encode or decode call for frame k, with $x \in \{\text{enc}, \text{dec}\}$. When each network runs *alone*, its mean time per frame over the N timed frames of a run is

$$\bar{t}_x = \frac{1}{N} \sum_{k=1}^{N} c_{x,k}, \qquad x \in \{\text{enc}, \text{dec}\}. \tag{1}$$

When the two take turns in one thread, the time per frame pair is

$$T_{\text{turns}} = \frac{1}{N} \sum_{k=1}^{N} \left(c^{\text{turns}}_{\text{enc},k} + c^{\text{turns}}_{\text{dec},k} \right), \tag{2}$$

where $c^{\text{turns}}_{x,k}$ are the call times measured in that mode. If switching between the two models had no cost, T_{turns} would equal $\bar{t}_{\text{enc}} + \bar{t}_{\text{dec}}$; the difference

$$\Delta_{\text{sw}} = T_{\text{turns}} - (\bar{t}_{\text{enc}} + \bar{t}_{\text{dec}}) \tag{3}$$

is therefore the *switching cost* per frame pair. When the two threads share the NE and each runs as fast as it can, each direction completes f_{shared} frames per second, and the time per frame pair is

$$T_{\text{shared}} = \frac{1}{f_{\text{shared}}}, \tag{4}$$

because in that time both directions advance by one frame. The *speedup* of sharing over taking turns is

$$S = \frac{T_{\text{turns}}}{T_{\text{shared}}}. \tag{5}$$

$S = 1$ means sharing gains nothing. If the NE could run the two networks fully in parallel, a pair would take only as long as the slower network, which gives the ideal speedup

$$S_{\max} = \frac{\bar{t}_{\text{enc}} + \bar{t}_{\text{dec}}}{\max(\bar{t}_{\text{enc}}, \bar{t}_{\text{dec}})}, \tag{6}$$

which is close to 2 when the two networks take similar times. Finally, the *contention factor* of each network is the ratio of its mean call time while sharing, $\bar{t}^{\text{shared}}_x$, to its mean time alone:

$$\kappa_x = \frac{\bar{t}^{\text{shared}}_x}{\bar{t}_x}, \qquad x \in \{\text{enc}, \text{dec}\}. \tag{7}$$

$\kappa_x = 1$ means the other network does not slow it down at all, and $\kappa_x = 2$ means it runs half as fast, as it would if the NE simply alternated between the two. Each of these quantities is computed both from the *call time*, which is the whole `libmlvc` encode or decode call including pre-processing and entropy coding, and from the *network time*, which is only the Core ML inference inside the call and is reported by `libmlvc` itself.

2.3 Real-Time Metrics

In a paced run the program behaves like a call at frame rate $f = 30$ fps. Frame k becomes *due* at

$$\tau_k = t_0 + \frac{k}{f}, \tag{8}$$

where t_0 is the common start time of the run. Let s_k and e_k be the times at which the call for frame k starts and ends. We define three times per frame:

$$c_k = e_k - s_k, \qquad \delta_k = s_k - \tau_k, \qquad \ell_k = e_k - \tau_k = \delta_k + c_k. \tag{9}$$

The *call time* c_k is the work itself; the *start delay* δ_k is how long the frame waited after it became due, for example behind the other direction or behind its own previous frame; and the *due-to-done time* ℓ_k is the delay the codec adds to the call. A frame is *late* if $\ell_k > D$, where $D = 1000/f = 33.3$ ms is the deadline. The encoder behaves like a camera: if it falls more than one frame period behind, the frames it missed are *dropped*, as a real camera does not wait. The decoder cannot drop frames, because every incoming frame depends on the previous one, so a slow decoder shows up as late frames instead. The share of late or dropped frames is

$$L = \frac{N_{\text{late}} + N_{\text{drop}}}{N_{\text{sched}}} \times 100\,\%, \tag{10}$$

where N_{sched} is the number of frames that became due during the timed part of the run, N_{late} the number with $\ell_k > D$, and N_{drop} the number dropped. Because a call is judged by its worst moments rather than its average, we report the 95th and 99th percentiles (p95, p99) of ℓ_k and c_k. For n sorted values $v_{(0)} \le \cdots \le v_{(n-1)}$, the p-th percentile is interpolated linearly at position $i = (n-1)\,p/100$:

$$P_p = v_{(\lfloor i \rfloor)} + (i - \lfloor i \rfloor)\left(v_{(\lfloor i \rfloor+1)} - v_{(\lfloor i \rfloor)}\right). \tag{11}$$

A p99 of 29 ms therefore means that 99% of frames finished within 29 ms of becoming due.

2.4 Energy

The power tool reports, for each sample j of duration Δ_j, the power P_j of each processor (CPU, GPU and NE). The mean power over a set of samples is weighted by duration:

$$\bar{P} = \frac{\sum_j P_j\,\Delta_j}{\sum_j \Delta_j}. \tag{12}$$

We compute $\bar{P}_{\text{run}}$ over the steady part of a run and $\bar{P}_{\text{idle}}$ over a separate idle recording with the laptop in the same state but no codec running. The energy that the call adds per frame pair is then

$$E_{\text{pair}} = \frac{\left(\bar{P}_{\text{run}} - \bar{P}_{\text{idle}}\right)\,T_{\text{w}}}{N_{\text{w}}}, \tag{13}$$

where T_{w} is the length of the steady window and N_{w} the number of frame pairs completed in it; at 30 pairs per second this equals $(\bar{P}_{\text{run}} - \bar{P}_{\text{idle}})/30$. The energy per minute of call is $E_{\text{min}} = (\bar{P}_{\text{run}} - \bar{P}_{\text{idle}}) \times 60$ s. Subtracting the idle power isolates the cost of the codec from the cost of keeping the laptop on.

2.5 Quality

For two 8-bit images X and $\hat{X}$ of M pixels, the mean squared error and the peak signal-to-noise ratio are

$$\text{MSE}(X,\hat{X}) = \frac{1}{M}\sum_{i=1}^{M}\left(X_i - \hat{X}_i\right)^2, \qquad \text{PSNR}(X,\hat{X}) = 10\log_{10}\frac{255^2}{\text{MSE}(X,\hat{X})}\ \text{dB}. \tag{14}$$

We compute it on the luma plane Y (PSNR-Y) and on the two chroma planes U and V, and combine them with the weighting customary in video coding,

$$\text{PSNR}_{\text{YUV}} = \frac{6\,\text{PSNR}_{\text{Y}} + \text{PSNR}_{\text{U}} + \text{PSNR}_{\text{V}}}{8}, \tag{15}$$

which reflects that the eye is far more sensitive to luma. These *coded-scale* values compare the decoded 360p frame with the 360p frame that was fed to the encoder, and measure only the distortion

of the codec. A viewer, however, sees the frame enlarged to the screen. We therefore also compute a *display-scale* PSNR-Y,

$$\mathrm{PSNR}_{\mathrm{disp}} = \mathrm{PSNR}\Big(\mathcal{U}(\hat{X}^{360}),\, X^{1080}\Big), \tag{16}$$

where $\hat{X}^{360}$ is the decoded frame, $\mathcal{U}$ is Lanczos upscaling to 1920×1080, and X^{1080} is the original 1080p frame; this includes both the coding loss and the loss of detail caused by the low resolution. *Face* PSNR-Y applies Eq. (16) only to the pixels inside the face boxes $\mathcal{B}$ found in the original frame. Inside the same boxes we compute LPIPS [10], a learned perceptual distance:

$$d_{\mathrm{LPIPS}}(X, \hat{X}) = \sum_{l} \frac{1}{H_l W_l} \sum_{h,w} \Big\| w_l \odot \Big(\phi^l_{hw}(X) - \phi^l_{hw}(\hat{X})\Big)\Big\|_2^2, \tag{17}$$

where $\phi^l_{hw}(\cdot)$ are the unit-normalised features of layer l of an AlexNet network at position (h, w), $H_l \times W_l$ is the size of that layer, w_l are learned per-channel weights, and $\odot$ is element-wise multiplication. LPIPS is 0 for identical images, grows as they differ in ways people notice, and is therefore read as "lower is better".

To compare two codecs over a range of bitrates we use the Bjøntegaard delta rate (BD-rate) [2]. For each codec, the logarithm of the bitrate $r = \log_{10} R$ is fitted as a cubic function of PSNR D through its four rate–distortion points, and

$$\text{BD-rate} = \left(10^{\frac{1}{D_H - D_L}\int_{D_L}^{D_H} (r_{\mathrm{test}}(D) - r_{\mathrm{ref}}(D))\,\mathrm{d}D} - 1\right) \times 100\,\%, \tag{18}$$

where $[D_L, D_H]$ is the PSNR range covered by both codecs, r_{test} belongs to MLVC and r_{ref} to the reference (anchor) codec. A BD-rate of −31% means MLVC needs on average 31% fewer bits than the anchor for the same PSNR.

Finally, to check whether the decoder gives the same pictures on every processor, we decode the same stream on two processors a and b and compute, for each frame k, $\mathrm{PSNR}(\hat{X}^a_k, \hat{X}^b_k)$ by Eq. (14). Identical decodes give infinite PSNR. We group the frames by their position $g = k \bmod 64$ after the most recent I-frame, because the carried state is reset at every I-frame and any difference can only build up between two I-frames.

3 Experimental Setup

3.1 The Baseline B^*

The baseline, which we call B^*, is a full-duplex call on one laptop in which both the encoder and the decoder of full MLVC run on the Neural Engine, inside one process, as two threads that work at the same time. The video is 640×360 at 30 fps, each frame has a deadline of 33.3 ms, the rate is fixed at QP 26, an I-frame is inserted every 64 frames, and one temporal layer is used. The content is talking-head video from scenario `s1` of the VCD dataset [5], and each run lasts 60 s, which is 1,802 frames in each direction. Table 2 lists these settings. QP 26 was chosen because it lies between the two middle points of the standard QP set (24 and 32) and gives bitrates of 120–290 kbps, which is typical of a call; an I-frame every 64 frames is the default of the full model.

3.2 Platform, Libraries and Tools

All experiments ran on one laptop, a 14-inch MacBook Pro with an Apple M4 Pro chip and 24 GB of unified memory, under macOS 26.6 (build 25G83). The M4 Pro has CPU cores, a GPU and a Neural Engine that share the same memory, which is what makes it a natural target for DuplexNVC. Running the whole baseline on one laptop removes the network and a second device from the measurement; running a call between two laptops is a later contribution.

Table 2. Baseline configuration B^*.

Setting	Value
Model	Full MLVC, `dmc61sbr_reglu`, weights `psnr_v1`, Core ML fp16
Resolution	640×360 video (model input 640×368)
Processors	Encoder on the NE, decoder on the NE
Execution	Full duplex: encoder thread and decoder thread in one process
Frame rate and deadline	30 fps, 33.3 ms per frame
Rate control	Fixed QP 26, I-frame every 64 frames, one temporal layer
Content	VCD [5] scenario `s1` (talking head)
Run length	60 s per run (1,802 frames per direction)

The codec is `libmlvc` version 0.1.0, at source commit `eb5d60a`. It is a C++ library that loads MLVC model bundles, creates encoder and decoder objects, and runs their networks through Apple's Core ML framework on the processor requested by the caller. Our measurement program links to it statically. We built both with CMake and the Ninja build system using Apple's Clang compiler from the Xcode Command Line Tools, in Release mode for the arm64 architecture. The large model files in the repository are stored with Git LFS.

The full MLVC model does not ship with `libmlvc`, so we converted it with the `mlvc-tools` package (`convert.py bundle`). This script loads the PyTorch checkpoint, traces the encoder and the decoder networks, converts them with Apple's `coremltools` into Core ML packages at fp16, checks them on a short test clip, and packs them into a bundle that `libmlvc` can load. The conversion ran under PyTorch 2.10 in a Python environment managed by `uv`.

Video was prepared with `ffmpeg`, which converted the 1080p MP4 references and the raw clips between the planar YUV 4:2:0 (I420) format of the dataset and the semi-planar NV12 format that `libmlvc` uses. The analysis is written in Python 3.13. NumPy reads the raw video files and computes PSNR and drift; OpenCV performs the Lanczos upscaling and the colour conversions for the display-scale metrics; the `lpips` package with PyTorch and torchvision computes LPIPS with the AlexNet backbone. Face boxes were found with Apple's Vision framework (`VNDetectFaceRectanglesRequest`) in a short Swift program. Power was recorded with Apple's `powermetrics` tool. BD-rate against hardware HEVC was computed by the `mlvc validate` command of `libmlvc`, which ships with the anchor's rate–distortion points. Table 3 summarises these components.

3.3 Measurement Conditions: Quiet Mode

Timing and energy on a laptop are easily disturbed by the operating system, so every timed run was made under the same conditions, which we call *quiet mode* (Table 4). Quiet mode is the reason the experiments are slow: while it is on, the laptop cannot be used for anything else, it has no network connection, and every run is followed by a rest of equal length. The laptop was on AC power, so that the chip was not limited by a battery-saving policy and the energy runs started from the same state. Wi-Fi was turned off during all timed and energy experiments, so that background network activity such as cloud synchronisation, software updates and indexing of downloaded files could not compete for the CPU or wake the system in the middle of a run; it was turned on only between experiments, to download data and to copy the logs off the laptop. The screen brightness was fixed at maximum and automatic brightness was not used, so that the display drew the same power in every run and in the idle recording, and the idle subtraction of Eq. (13) removed

Table 3. Platform, libraries and tools.

Component	What we used
Device	MacBook Pro 14-inch, Apple M4 Pro (Mac16,8), 24 GB unified memory
Operating system	macOS 26.6 (build 25G83)
Codec library	`libmlvc` 0.1.0, commit `eb5d60a`, Core ML back end
Build	CMake, Ninja, Apple Clang (Command Line Tools); Release, arm64, static; Git LFS
Model conversion	`mlvc-tools` (`convert.py`), PyTorch 2.10, `coremltools`, `uv`
Measurement program	C++17: `std::thread`, `std::chrono::steady_clock`, `sleep_until`
Video conversion	`ffmpeg` (MP4 and I420 to NV12)
Analysis	Python 3.13, NumPy, OpenCV (Lanczos), `lpips` with PyTorch and torchvision (AlexNet)
Face detection	Swift, Apple Vision (`VNDetectFaceRectanglesRequest`)
Power	`powermetrics`, samplers `cpu_power` and `gpu_power` (the latter includes the NE)
BD-rate	`mlvc validate` with the bundled Intel hardware HEVC anchor

it completely; maximum was chosen because it is a setting that can be reproduced exactly. The command `caffeinate -dims` kept the display, the disk and the system from sleeping during long runs. No other user application was open. Unpaced runs were interleaved in rounds (taking turns, encoder alone, decoder alone, sharing), so that slow drifts in the state of the laptop affect every mode equally, and every 60 s run was followed by 60 s of rest, so that heat from one run did not carry into the next. Time Machine backups could not be paused, because the command needs Full Disk Access; the spread between repeated runs (below 1% for timing and 2% for energy) shows that background activity had little effect. For the extension of Section 7 we also turned Bluetooth off and disabled True Tone, recorded the thermal state of the chip and the Low Power Mode setting once per second, and recorded the output of `pmset -g therm` with every run, so that any slowdown caused by heat would be visible in the records.

Table 4. Quiet mode: the state of the laptop during every timed and energy run.

Setting	Value and reason
Power	AC adapter connected; no battery-saving policy can limit the chip
Wi-Fi	Off during every run; on only between experiments to copy data. No cloud sync, updates or indexing
Bluetooth	Off (extension runs)
Screen brightness	Fixed at maximum; automatic brightness and True Tone off, so the display draws the same power in every run and in the idle recording
Sleep	`caffeinate -dims`: display, disk and system kept awake
Other software	No other user application open; Low Power Mode off
Rest	60 s idle after every 60 s run, so heat does not carry over
Order	Modes interleaved in rounds; first 64 frames of each direction discarded
Monitoring	Thermal state and Low Power Mode logged at 1 Hz; `pmset -g therm` per run (extension)

3.4 The Measurement Program

We considered three ways to measure the baseline: the `mlvc` command-line tool alone, a small C++ program linked to `libmlvc`, and a Swift application. We chose the C++ program. It uses exactly the library that ships, so its numbers are those of the real codec, yet it gives us control over threads, pacing and per-frame timestamps, which the command-line tool does not expose. The command-line tool is kept as an independent check of our numbers (experiment E2).

The program, `duplex_bench` (about 380 lines of C++17), works as follows. At start-up it reads the whole camera clip, a raw NV12 file, into memory, so that disk reads never fall inside a timed call, and it reads the incoming stream, a file of MLVC access units. It then creates one `libmlvc` manager per processor in use, with the model cache enabled and all model sessions loaded at initialisation, and from it an encoder configured for 640×360, QP 26 and an I-frame period of 64, and a decoder. The first encode and decode load the models onto the processor and take far longer than a normal frame, so the program performs one encode and one decode before the clock starts; the I-frame produced by this warm-up encode is kept, because every later frame of the outgoing stream depends on it. It then sets a common start time t_0 200 ms in the future and every mode waits for it.

The program has five modes. `prepare` encodes the incoming clip once, untimed, and writes it as an MLVC stream, so that the decoder in later runs receives a real stream from "the other participant". `alone-enc` and `alone-dec` run only the encoder or only the decoder, and give $\bar{t}_{\text{enc}}$ and $\bar{t}_{\text{dec}}$ of Eq. (1). `turns` runs one thread that encodes frame k and then decodes frame k, and gives T_{turns} of Eq. (2). `duplex` starts two `std::thread` objects, one looping over encodes and one over decodes, and gives T_{shared} of Eq. (4) and the baseline B^*.

Every mode runs either *unpaced* or *paced*. Unpaced (`-fps 0`), each frame starts as soon as the previous one ends, which measures how fast the codec can go. Paced, frame k is due at τ_k of Eq. (8), and the thread sleeps with `std::this_thread::sleep_until` until that moment before starting it, which measures how the codec behaves in a real call. In paced duplex mode the encoder thread checks before each frame whether it has fallen more than one period behind; if so, it skips to the newest due frame and counts the skipped frames as dropped, like a camera. The decoder thread never skips. All times are read from `std::chrono::steady_clock`, a monotonic clock with sub-microsecond resolution that is not affected by changes to the system time.

For every frame the program records one row in a file `frames.csv`: the direction, the frame number, the source frame, the due, start and end times τ_k, s_k and e_k relative to t_0, the call time, the network time reported by `libmlvc`, the frame type (I or P), the size in bytes, whether the call succeeded, and whether the frame belongs to the warm-up. It also writes the outgoing stream, so that it can be decoded later on other processors, and a summary file with the settings, the number of dropped and failed frames, and the wall-clock time of t_0, which is used to align the power log with the run. Figure 1 shows the duplex mode.

3.5 Analysis of the Records

A Python script reads `frames.csv` and computes every timing quantity of Sections 2.2 and 2.3. It first removes the first 64 frames of each direction (about 2 s), during which caches and processor clocks settle; the remaining 1,738 frames per direction form the timed part of a run. It then computes means, the percentiles of Eq. (11), the late and dropped share of Eq. (10), and, from separate runs of the four modes in the same round, S, $S_{\max}$ and κ_x. For each quantity we report the median over the repeated runs, which is robust to one unusual run, together with the minimum and maximum, which show how repeatable the run was.

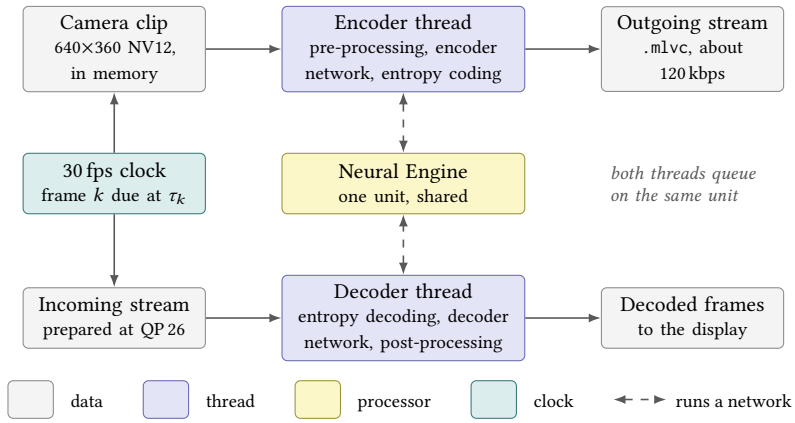


Fig. 1. Duplex mode of the measurement program. One 30 fps clock makes a camera frame and an incoming frame due at the same instant; the encoder thread and the decoder thread then both run their networks on the one Neural Engine. In `turns` mode a single thread runs the two steps one after the other.

For energy, `powermetrics` was started with the `cpu_power` and `gpu_power` samplers, which report CPU, GPU and Neural Engine power about every 125 ms. Before the energy runs we recorded 120 s with the laptop idle under the same conditions. The script keeps only the samples whose time stamps fall inside the timed part of a run, one second inside each end, which leaves a steady window of about 56 s, and applies Eqs. (12) and (13).

For quality, each clip was encoded and decoded at QP 16, 24, 32 and 40. The script computes coded-scale PSNR with NumPy, upscales every decoded frame to 1080p with OpenCV's Lanczos filter and compares it with the 1080p original, and on every 10th frame computes face PSNR-Y and LPIPS inside the boxes found by the Swift face detector on the original frame. For drift, the same outgoing stream was decoded with the `mlvc` tool on the NE twice and on the GPU and CPU once each, and the decodes were compared frame by frame as described in Section 2.5.

3.6 Data

The baseline in this report uses scenario `s1` of the VCD video conferencing dataset [5], which contains talking-head clips of 10 s at 30 fps. We used it in three sets with three different jobs.

The *timing streams* use the four `s1` clips that ship with `libmlvc`. The camera stream joins clips `0380a3` and `6587ae`, and the incoming stream joins `6c9c96` and `9c02f6`; each has 600 frames (20 s) and is looped over a 60 s run. The incoming stream was prepared once at QP 26, with a PSNR-Y of 34.98 dB and a PSNR-YUV of 37.36 dB. The incoming clips have more motion and detail than the camera clips, so the two directions carry different bitrates. For the extension (Section 7) the same four clips were scaled from their 1080p releases to 540p, 720p and 1080p.

The *quality set* consists of 40 `s1` clips that we downloaded at 360p as raw YUV, together with their 1080p releases as MP4 files, which serve as the display reference. We split them at random, with seed 694, into 20 calibration clips and 20 evaluation clips, forcing the four shipped clips into the calibration set so that no clip used for timing is also scored for quality. The calibration set is reserved for tuning DuplexNVC later; all quality results in this report come from the 20 evaluation clips.

The *BD-rate set* is the validation set of `libmlvc`: three 3 s `s1` clips at 640×360, with the rate–distortion points of an Intel hardware HEVC encoder in low-power mode as the anchor.

3.6.1 Data for the Controller Study. The three sets above serve the fixed-MLVC baseline. The controller of DuplexNVC (Section 9.2) also needs clips that vary in the number of people, their face size and whether people enter or leave the frame. We plan to use three sources, summarised in Tables 5 and 6; preparing them is step N6 of the plan (Section 9.1).

Full VCD.. We extend from the 40-clip subset to all 160 clips of VCD [5]: four scenarios of 40 clips each, covering desktop webcams, two kinds of background processing and handheld mobile recordings. Each clip is 10 s of 1080p, YUV 4:2:0 video at 30 fps (300 frames). The raw size is about 0.93 GB per clip, or about 150 GB in total. VCD shows one talking head per clip, so it covers the single-person and small-face cases across camera qualities but not groups. We keep the earlier split: the 20 calibration clips of the subset, which include the four shipped `libmlvc` clips, stay in calibration, and its 20 evaluation clips stay in evaluation. The remaining 120 clips are split at random with seed 694, balanced by scenario, so that each side holds 80 clips (20 per scenario). The fixed-MLVC baseline will be re-run on the same evaluation clips so that every controller comparison uses identical content.

HEVC Class E.. The three conferencing sequences of the JCT-VC common test conditions [7], *Johnny*, *KristenAndSara* and *FourPeople*, show one, two and four people in a fixed-camera meeting setting. Each is 1280×720, 8-bit YUV 4:2:0 at 60 fps with 600 frames (831 MB as `.y4m`). They are taken from the Xiph.org test media collection. We keep every second frame, which gives 300 frames at 30 fps with no duplicated frames. Because these sequences are standard in codec studies, they also anchor our whole-frame BD-rate results, and the on-device HEVC anchor of step N7, to content that other work reports on.

Pexels multi-person clips. To cover groups and changes in the number of people, we use eight clips from Pexels [8], which are free to use and modify. They show 4, 5 and 7 people around a table, and people entering and leaving the frame, filmed with a fixed camera. They are H.264 MP4 files of 10.7 to 45.8 MB. Most are 25 fps; we retime them to 30 fps by changing the timestamps (`setpts=PTS*25/30`), so every original frame is kept and none is duplicated or interpolated, and decode them to YUV 4:2:0 `.y4m`. Because these sources were already compressed once, we report their face-quality results separately from VCD and Class E. Their resolution and frame count after retiming will be recorded in Table 6 when they are converted.

Preprocessing and test inputs. Every clip is downscaled with the Lanczos filter of `ffmpeg`, as for the baseline, to the processing resolutions of MLVC used in this report (360p, 540p and 720p) for coding. Face boxes are detected once on the source frames and reused for every method, so face-region PSNR and LPIPS (Section 2.5) are measured on the same regions and at the same display scale. The incoming direction replays one prerecorded MLVC bitstream, as in the timing streams, and the peer's energy mode follows one scripted trace; both are identical for every controller. For the 30-minute sustained-load scenario of step N4, the camera and incoming streams are looped evaluation clips, as in the timing streams.

Access. VCD is available from its GitHub repository (https://github.com/microsoft/VCD), the Class E sequences from the Xiph.org test media collection (https://media.xiph.org/video/derf/), and the Pexels clips under the Pexels license; the page URL of each clip is listed in our repository (Section 3.9).

Programming model. The baseline measurements use the C++ program of Section 3.4. The DuplexNVC controller pipeline is planned in Swift: both codec directions run as Core ML models, scene analysis uses Apple Vision, and the encode and decode directions run concurrently on

separate Grand Central Dispatch queues. Any controller will be measured with the same conditions and metrics as this report.

Table 5. Required cases for the controller study and the data that covers them.

Required case	**Covered by**
Talking-head conferencing content	Full VCD, 160 clips
1 person	VCD; *Johnny*
2 people	*KristenAndSara*; Pexels `2_people_entering_room`
4 people	*FourPeople*; Pexels `4_people`
5–7 people	Pexels `5_people` and `7_people`
Different distances	VCD (desktop and mobile); Pexels room views
People entering and leaving	Pexels: 3 entering clips, 2 leaving clips
Separate calibration and evaluation clips	Table 6, last column
Same incoming workload for every controller	One prerecorded incoming bitstream and one peer-energy trace
Sustained 30-minute call	Looped evaluation clips

Table 6. Datasets planned for the controller study. Frame counts are after conversion to 30 fps; sizes are raw for VCD and Class E and compressed (MP4) for Pexels, whose resolution and length are recorded when they are converted (–).

Clip(s)	**Source**	**People**	**Source format**	**Frames**	**Size**	**Role**
VCD, 80 clips	VCD [5]	1	1080p30 YUV	300 each	0.93 GB each	Calibration
VCD, 80 clips	VCD [5]	1	1080p30 YUV	300 each	0.93 GB each	Evaluation
Johnny	Class E [7]	1	720p60 YUV	300	831 MB	Calibration
KristenAndSara	Class E [7]	2	720p60 YUV	300	831 MB	Evaluation
FourPeople	Class E [7]	4	720p60 YUV	300	831 MB	Evaluation
`4_people`	Pexels [8]	4	H.264 MP4	–	45.5 MB	Calibration
`5_people`	Pexels [8]	5	H.264 MP4	–	12.4 MB	Calibration
`7_people`	Pexels [8]	7	H.264 MP4	–	10.7 MB	Evaluation
`people_entering`	Pexels [8]	varies	H.264 MP4	–	29.5 MB	Calibration
`people_entering_room`	Pexels [8]	varies	H.264 MP4	–	30.2 MB	Evaluation
`2_people_entering_room`	Pexels [8]	2	H.264 MP4	–	45.8 MB	Evaluation
`people_leaving_going_downstairs`	Pexels [8]	varies	H.264 MP4	–	19.6 MB	Calibration
`people_leaving`	Pexels [8]	varies	H.264 MP4	–	42.7 MB	Evaluation

3.7 Experiments and Protocol

Table 7 lists every experiment. Two smoke tests, S1 for MLVC-S and S2 for the baseline, ran each mode once for 10 s to check the pipeline. E3a repeats the Phase 1 MLVC-S measurements in five rounds of 60 s runs plus three paced runs. The baseline experiments all use full MLVC at 360p: E1 measures each network alone; E2 runs the `libmlvc` command-line benchmark three times as a check; E3 and E4 measure taking turns and sharing, unpaced, in five rounds; B^* itself is five paced duplex runs of 60 s, with three paced taking-turns runs for reference; E5 decodes one B^* stream on

every processor; E6 measures compression quality on the three sets of Section 3.6; and E7 measures the energy of five B^* runs after a 120 s idle recording.

Table 7. Experiments reported.

ID	Model	What is measured	Repetitions
S1	MLVC-S 540p	Smoke test: every mode runs, the NE is used	1×10 s per mode
S2	Full MLVC 360p	Smoke test of the baseline pipeline	1–2×10 s per mode
E3a	MLVC-S 540p	Check of Phase 1: alone, turns, shared; paced turns	5 rounds; 3 paced
E1	Full MLVC 360p	Encoder and decoder alone, against the MLVC paper	5 rounds
E2	Full MLVC 360p	`libmlvc` command-line duplex benchmark	3 runs
E3, E4	Full MLVC 360p	T_{turns}, T_{shared}, S, κ_x, unpaced	5 rounds
B^*	Full MLVC 360p	Duplex paced at 30 fps	5×60 s
–	Full MLVC 360p	Taking turns paced at 30 fps, for reference	3×60 s
E5	Full MLVC 360p	Decoding correctness and drift across processors	1 stream, 4 decodes
E6	Full MLVC 360p	BD-rate; rate–distortion, display and face quality	3, 20 and 4 clips
E7	Full MLVC 360p	Energy of B^* on AC power	120 s idle, 5 runs
Extension (Section 7)			
X1	Full MLVC 360p, MLVC-S 540p	Operation placement trace; each network alone on NE, GPU and CPU, unpaced and paced; power	5 or 3 rounds; 3×60 s paced
X2	Seven settings	Conversion, placement trace, input streams, incoming streams at QP 26	once
X3	Seven settings, multi-size	22-hour run: NE sanity against the survey, memory, each network alone, every placement unpaced and paced at 30 (and 15) fps, energy	3 rounds; 5×60 s paced

3.8 Measurement Effort

Table 8 lists how long each campaign kept the laptop busy. The times are wall-clock times from the first to the last run of each campaign and include the rests of the protocol; they exclude installing tools, building the program and copying data. Together the campaigns kept the laptop in quiet mode for about 29 hours; with model conversion, quality scoring, building and set-up, the work took more than 30 hours of laptop time.

Table 8. Time the laptop spent on each campaign.

Campaign	Duration
Baseline B^*: S1, S2, E1–E7	about 5.3 h
X1: trace, processors alone (72 runs)	about 2.3 h
X2: conversion and input streams	about 0.3 h
X3: full experiment matrix (584 runs)	21.4 h (1,283 min)
Total	**about 29 h**

3.9 Reproducibility and Experiment Scripts

All code, scripts, raw records and analysis of this report are kept in our GitHub repository, https://github.com/ShamsAlani/DuplexNVC, organised by experiment, with a README in every folder that explains how to reproduce it. The repository is private; access is available on request.

The program was built with the following commands, where `$WORK` is the project folder:

```
cmake -S . -B build/ninja-release -G Ninja \
      -DCMAKE_BUILD_TYPE=Release -DCMAKE_OSX_ARCHITECTURES=arm64 \
      -DBUILD_SHARED_LIBS=OFF \
      -DFETCHCONTENT_SOURCE_DIR_LIBMLVC=$WORK/libmlvc
cmake --build build/ninja-release
```

The full MLVC bundle was produced with:

```
uv run convert.py bundle --model-version dmc61sbr_reglu \
   --weights-path ~/mlvc/job-outputs/pretrained/mlvc-psnr-v1.ckpt \
   --weights-version psnr_v1 --model-input-sizes 640x368 \
   --mlvc-version v0.1 ...
```

The program is called as `duplex_bench <mode>`, with the mode one of `prepare`, `alone-enc`, `alone-dec`, `turns` or `duplex`, and the options `–input`, `–size`, `–incoming`, `–bundles`, `–enc-cu`, `–dec-cu`, `–qp`, `–iframe-period`, `–fps` (0 means unpaced), `–deadline-ms`, `–seconds`, `–warmup-frames` and `–out-dir`. Short shell scripts drive the experiments, each saving its console output to a text file; Table 9 lists them. Each script runs one experiment of Table 7 from start to end under the conditions of Table 4, writes one folder per run with its `frames.csv`, summary file, outgoing stream and power log, and is followed by a summary script that computes every number reported in Sections 4 to 7 from these records alone. The scripts of the extension are resumable: if a run is interrupted, the script can be started again and skips every run whose records are complete, which is what allowed X3 to run for almost 22 hours unattended. Every table can therefore be regenerated from the raw records in the repository with these summary scripts.

Table 9. Experiment scripts.

Script	Purpose
`p`	Prepare the Mac (tools and environment)
`s`, `e`, `r`	MLVC-S 540p smoke test (S1), experiment E3a and its summary
`b`	Rebuild the program
`m`	Build the 360p camera and incoming streams; prepare the incoming stream at QP 26
`t`	Baseline smoke test (S2)
`x`	B^* experiments: 5 unpaced rounds, 5 paced duplex runs, 3 paced turns runs, 3 command-line runs (E1–E4)
`y`	Summary of `x`
`f`, `g`	Download the 1080p references; split, rate–distortion and face quality on the evaluation set (E6)
`q`	`mlvc validate` BD-rate and rate–distortion on the shipped clips (E6)
`w`	Energy (E7)
`c`	Correctness and drift (E5)
`u`	Restore system settings
Extension (Section 7)	
`mlplan.swift`, `1t`	Operation placement trace with Core ML's `MLComputePlan` (X1, X2)
`1`, `1r`	Each network alone on NE, GPU and CPU, unpaced and paced, with power (X1), and its summary
`s2cfg`	Shared settings of the extension: paths, clips, the seven settings
`2a`	Conversion of the six new bundles and the multi-size bundle, and their trace (X2)
`2b`	Camera and incoming streams per resolution; incoming at QP 26 and decode check (X2)
`thermal_log.swift`	Thermal state and Low Power Mode at 1 Hz
`all`	The 22-hour experiment X3, resumable
`2r`, `4r`	Summaries of X3: NE sanity against the survey; table of feasible settings

4 Smoke Tests and Bring-Up

Before collecting any data we ran each configuration for 10 s in every mode. The aim was to check the pipeline, not to produce results, and a configuration passed if four conditions held: no call failed, Core ML placed the networks on the NE (the library reported `requested=npu resolved=npu`), every run ended on its own, and paced runs kept up with 30 fps. Table 10 lists the outcome.

Both configurations passed. For MLVC-S at 540p (S1), the encoder and decoder alone took 4.86 and 4.92 ms, taking turns took 9.82 ms per pair and sharing 8.53 ms, a speedup of 1.15, and paced taking turns had no late frame. For full MLVC at 360p (S2), the encoder and decoder alone took 5.90 and 5.96 ms, taking turns 11.86 ms and sharing 10.68 ms, a speedup of 1.11; paced taking turns had no late frame, and paced duplex had no late decoded frame and one late encoded frame out of about 300 (0.37%). That one frame did not recur in any of the five 60 s baseline runs (Section 6.4), so we read it as a start-up effect of the short run rather than a property of the baseline. These short runs already agree with the full experiments to within 1%, which shows that the program measures consistently. The first encode and decode took 6.6–15.9 ms, two to three times a normal

frame, which confirms that the warm-up before t_0 is needed: without it, the first frames of every run would start late and appear as a backlog.

Table 10. Smoke-test results (single 10 s runs; indicative only).

Quantity	**S1: MLVC-S 540p**	**S2: full MLVC 360p**
Encoder alone, call time (ms)	4.86	5.90
Decoder alone, call time (ms)	4.92	5.96
T_{turns} (ms)	9.82	11.86
T_{shared} (ms)	8.53	10.68
S	1.15	1.11
Duplex paced, late or dropped, send / receive	–	0.37% / 0%
Turns paced, late or dropped, send / receive	0% / 0%	0% / 0%
First frame, encoder / decoder (ms)	8.5–13.4 / 6.6–13.8	10.5–15.9 / 7.9–10.2
Failures; processor used	0; NE	0; NE

Bringing the pipeline up also exposed several problems, listed with their fixes in Table 11. The most important was in our own program: in unpaced mode, the schedule was still computed from t_0 and the frame period, which is zero when unpaced, so the end condition was never reached and runs never ended. We changed unpaced frames to be scheduled at the moment they start, made every mode wait for t_0, and moved the warm-up before t_0. The others were problems of the environment: the Xcode build generator was not available because only the Command Line Tools were installed, so we switched to Ninja; an interrupted clone of the repository was repaired by cloning again without downloading the large files and fetching them afterwards; the conversion of full MLVC needed a test clip, which we produced with `ffmpeg`; the LPIPS weights failed to download because of a certificate error, so we placed them in the PyTorch cache by hand; and the energy script waited for a background helper that never ended, which we fixed by waiting only for the measured process. Finally, we confirmed that the baseline uses full MLVC at 360p, as agreed, and kept the MLVC-S runs only as a check of Phase 1.

Table 11. Problems found during bring-up and their fixes.

Problem	**Fix**
Unpaced runs never ended	Unpaced frames are scheduled at the current time instead of $t_0 + k \cdot$period; every mode waits for t_0; the warm-up runs before t_0, and its I-frame is kept so the outgoing stream stays decodable.
CMake Xcode generator missing	Built with Ninja (Release, arm64, static) against the local `libmlvc` copy.
Repository clone interrupted	Cloned again with `GIT_LFS_SKIP_SMUDGE=1`, then `git lfs pull`.
Conversion needs a test clip	Made it from `libmlvc`'s NV12 test clip with `ffmpeg` (NV12 to I420).
LPIPS weights failed to download	Placed the AlexNet weights in the PyTorch cache by hand.
Energy script did not return	`wait` also waited on the `sudo` keep-alive loop; replaced with `wait $!`.
Which model is the baseline	Baseline is full MLVC at 360p; MLVC-S 540p (E3a) kept only as a check of Phase 1.

5 Check of Phase 1: MLVC-S at 540p (E3a)

Phase 1 reported MLVC-S times of which two were *estimated* rather than measured: T_{turns} was taken to equal $\bar{t}_{\text{enc}} + \bar{t}_{\text{dec}}$, that is, the switching cost of Eq. (3) was assumed to be zero, and the speedup S was inferred. E3a measured both with the new program, under the conditions of Section 3.3, in five interleaved rounds of 60 s runs and three paced runs (Table 12).

The encoder and decoder alone took 4.84 and 4.91 ms, against 5.0 ms each in Phase 1, so their sum is 9.75 ms against the estimated 10.0 ms. The measured T_{turns} is 9.79 ms, so switching between the two models costs only 0.04 ms per pair and the Phase 1 assumption was right. Sharing took 8.53 ms per pair (117.2 fps per direction), against 8.7 ms (115.5 fps) in Phase 1, and the measured speedup is exactly the estimated 1.15. The contention factors are 1.74 for the encoder and 1.71 for the decoder, close to the estimated 1.8. Every Phase 1 value therefore holds to within 4%, and the minimum and maximum of the five rounds differ by at most 0.02 ms, so the measurement is highly repeatable.

Paced at 30 fps, taking turns had no late or dropped frame. The encoder call took 10.69 ms and the decoder call 8.68 ms on average, about twice as long as unpaced; the 99th percentile of the due-to-done time was 18.9 ms for the encoder and 28.2 ms for the decoder, which is longer because the decoder always waits for the encoder in this mode. We conclude that the new program reproduces the method of Phase 1 and can be trusted for the new baseline, and we note that pacing doubles the call time, an effect that reappears in the baseline and is discussed in Section 6.4.

Table 12. E3a: MLVC-S 540p on the NE, median (min–max) of 5 rounds, against the Phase 1 values.

Metric	Phase 2 (measured)	Phase 1
$\bar{t}_{\text{enc}}$ alone, call time (ms)	4.84 (4.84–4.86)	5.0
$\bar{t}_{\text{dec}}$ alone, call time (ms)	4.91 (4.91–4.92)	5.0
$\bar{t}_{\text{enc}} + \bar{t}_{\text{dec}}$ (ms)	9.75	10.0 (estimate)
T_{turns}, measured (ms)	9.79 (9.79–9.80)	–
Switching cost Δ_{sw} (ms)	0.04	–
T_{shared} (ms)	8.53	8.7
Frames per second per direction, shared	117.2	115.5
S	1.15	about 1.15 (estimate)
κ_{enc} / κ_{dec} (network time)	1.74 / 1.71	about 1.8
Turns paced: mean call, send / receive (ms)	10.69 / 8.68	–
Turns paced: due-to-done p99, send / receive (ms)	18.9 / 28.2	–
Turns paced: late or dropped	0% / 0%	–

6 Baseline Results: Full MLVC, 360p, NE

6.1 E1: Each Network Alone

Running alone on the NE, the full MLVC encoder took 5.86 ms per frame (170.7 fps) and the decoder 5.94 ms (168.4 fps), with a spread of only 0.06 ms over five rounds (Table 13). Of this, the networks themselves took 5.57 and 5.66 ms, so everything outside the networks, namely colour conversion, padding, entropy coding and the library's bookkeeping, adds only 0.29 ms to an encode and 0.28 ms to a decode, about 5% of the call.

The MLVC paper reports 7.72 ms (129.5 fps) for the encoder and 8.14 ms (122.9 fps) for the decoder on an M3 Pro at 360p [6]. Our times are 24–27% shorter. This is what one expects from the newer Neural Engine of the M4 Pro, and it shows that our own conversion of the full model runs as fast as the authors' and was not slowed down by, for example, operations falling back to the CPU. Two things follow. First, each network alone needs less than a fifth of the 33.3 ms frame time, so on paper the NE has plenty of capacity for a call. Second, because the networks take 95% of the call, any speed-up must come from how the *networks* are run and placed, not from the code around them.

Table 13. E1: full MLVC 360p alone on the NE, median (min–max) of 5 rounds.

	Encoder	Decoder
Call time (ms)	5.86 (5.85–5.91)	5.94 (5.92–5.94)
Network time (ms)	5.57	5.66
Time outside the network (ms)	0.29	0.28
Frames per second	170.7	168.4
MLVC paper, M3 Pro, 360p [6] (ms)	7.72 (129.5 fps)	8.14 (122.9 fps)

6.2 E3 and E4: Taking Turns Against Sharing the NE

This experiment answers the central question of the baseline: can the NE run the encoder and the decoder in parallel? Table 14 gives the unpaced results, Figure 2 compares them with MLVC-S, and Figure 3 compares the measured speedup with its ideal.

The sum of the two networks alone is $\bar{t}_{\text{enc}} + \bar{t}_{\text{dec}} = 11.80$ ms, and taking turns measured $T_{\text{turns}} = 11.81$ ms. The switching cost of Eq. (3) is therefore 0.01 ms, which is within the measurement noise: alternating between two loaded Core ML models on the NE costs nothing. Sharing measured $T_{\text{shared}} = 10.68$ ms per pair, which is 93.6 fps in each direction. The speedup of Eq. (5) is thus $S = 11.81/10.68 = 1.11$, while the ideal of Eq. (6) is $S_{\max} = 11.80/5.94 = 1.99$. In other words, running both networks at once saves 1.13 ms of the 5.87 ms that perfect parallelism would save, about 19% of the possible gain. The contention factors of Eq. (7) explain why: while sharing, the encoder's call takes 1.82 times and the decoder's 1.80 times as long as alone (1.86 and 1.84 for the network time alone). A factor of 2 would mean that the NE simply alternates between the two networks; a factor of 1.8 means it overlaps them only slightly. All five rounds gave the same values to within 0.02 ms.

Figure 2 shows that MLVC-S at 540p behaves the same way ($S = 1.15$, $\kappa \approx 1.7$), even though it is a different network at a different resolution. The limitation therefore lies in the Neural Engine, not in a particular model. We deduce that the NE behaves almost as a single serial unit, and that the speedup DuplexNVC needs cannot come from running both networks in parallel on it. It has to come from placing work on processors that are idle (the GPU and the CPU), from cheaper models or resolutions, or from scheduling the two directions so that they compete less.

Table 14. E3 and E4: full MLVC 360p, unpaced, median of 5 rounds (all ranges within ±0.02 ms).

Metric	**Value**
$\bar{t}_{\text{enc}} + \bar{t}_{\text{dec}}$	11.80 ms
T_{turns}, measured (Eq. (2))	11.81 ms
Switching cost Δ_{sw} (Eq. (3))	0.01 ms
T_{shared} (Eq. (4))	10.68 ms (93.6 fps per direction)
Speedup S (Eq. (5))	**1.11**
Ideal speedup $S_{\max}$ (Eq. (6))	1.99
κ_{enc} / κ_{dec}, call time (Eq. (7))	1.82 / 1.80
κ_{enc} / κ_{dec}, network time	1.86 / 1.84

6.3 E2: Check Against the `libmlvc` Command-Line Tool

To make sure our program does not distort the numbers, we ran the benchmark built into the `mlvc` command-line tool in its own duplex setting, with one encoder stream and one decoder stream, three times for about 5,600 frames each. In every run it reported 10.7 ms per frame, of which 10.4 ms was inference, and 93.6 fps for both the encoder and the decoder stream. This is the same as our $T_{\text{shared}} = 10.68$ ms and 93.6 fps per direction, so our program adds no measurable cost or bias, and its results can be compared with those of the library's own tool.

6.4 B^*: Real-Time Duplex at 30 fps

The unpaced experiments measure capacity; the baseline itself is a call paced at 30 fps. Table 15 gives the five paced duplex runs and, for reference, three paced taking-turns runs, and Figure 4 compares their latencies with the deadline.

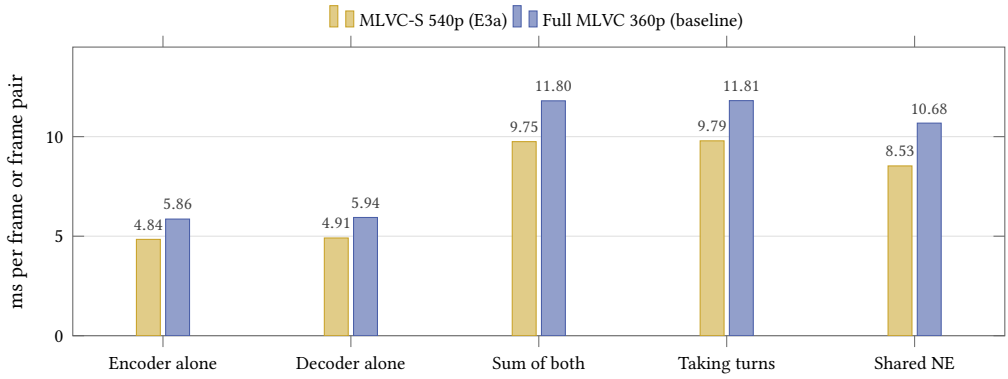


Fig. 2. Unpaced times on the NE. For both models, taking turns costs exactly the sum of the two networks, and sharing the NE recovers only a small part of it.

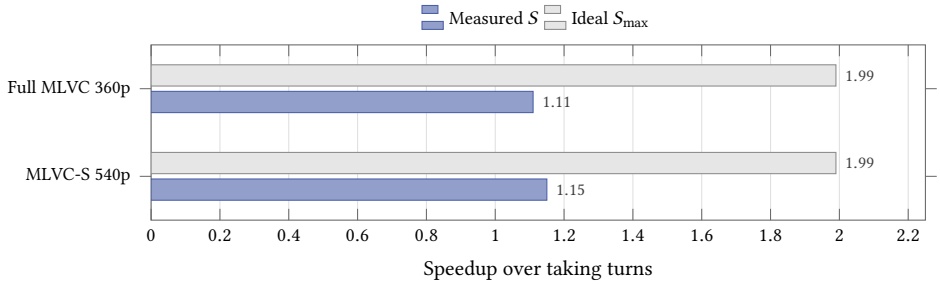


Fig. 3. Measured speedup of running both networks at once on the NE, against the ideal for perfect overlap.

Over five runs of 60 s, B^* delivered 30.01 fps in both directions with no late frame, no dropped frame and no failed call. The mean call time was 12.44 ms for the encoder and 13.32 ms for the decoder, and 99% of calls took less than 20.0 and 20.9 ms. The due-to-done time of Eq. (9), which adds the waiting before each call, reached 27.9 and 28.0 ms at the 95th percentile and 29.3 and 28.9 ms at the 99th. Against the deadline of 33.3 ms, the slowest 1% of frames therefore keep only about 4 ms, or 12%, of slack. The five runs differ by less than 0.3 ms in every percentile, so this margin is a property of the baseline, not of one run. The encoder produced 120.5 kbps and the incoming stream carried 287.1 kbps at the same QP; the difference is due to the content, since the incoming clips move more and have more detail. I-frames and P-frames took about the same time (12.15 against 12.43 ms for the encoder), so the periodic I-frame does not cause a spike.

Two observations explain why a system that can serve 93.6 fps per direction unpaced, about three times what a call needs, ends up so close to the deadline at 30 fps. First, a paced call takes about twice as long as an unpaced one: 12.4 ms here against 5.9 ms for the encoder alone, and the same doubling appeared for MLVC-S in E3a. This is consistent with the NE lowering its clock during the idle part of each frame period and having to speed up again at the start of every frame, although we have not measured the clock directly. Second, the camera frame and the incoming frame become due at the same instant, so one of the two calls waits for the other on the shared NE: the start delay δ_k is 5–6 ms on average and reaches 10 ms at the 95th percentile. The slowest frames are those that wait the longest and then run the slowest. We deduce that any heavier setting, such

as a higher resolution, a larger model or a second remote participant, is likely to miss deadlines on the NE alone unless this contention is reduced, and that offsetting the two directions in time is a cheap way to reduce the waiting.

Taking turns at 30 fps also met every deadline, and its encoder finished sooner (p99 21.3 ms), because the encoder always runs first. The decoder, however, always waits for the encoder, with a mean start delay of 14.5 ms, and its p99 reached 31.1 ms, closer to the deadline than either direction of B^*. Sharing the NE therefore does not make the work faster in total, but it spreads the waiting over both directions and keeps both of them further from the deadline, which is why it is the baseline.

Table 15. Baseline B^* (duplex, paced at 30 fps), median (min–max) of 5 runs of 60 s, and taking turns paced at 30 fps (3 runs) for reference.

	B^*, shared NE		Taking turns	
Metric	**Send**	**Receive**	**Send**	**Receive**
Frames per second	30.01	30.01	30.01	30.01
Late or dropped, L	**0%**	**0%**	0%	0%
Mean call time (ms)	12.44 (12.39–12.47)	13.32 (13.22–13.40)	10.66	8.44
p99 call time (ms)	20.03	20.91	13.15	9.99
Due-to-done p95 (ms)	27.88	28.02	20.17	29.81
Due-to-done p99 (ms)	**29.27** (29.12–29.33)	**28.93** (28.66–29.16)	21.30	31.06
Start delay, mean / p95 (ms)	5.79 / 10.00	5.08 / 10.00	3.84 / 7.50	14.50 / 20.10
Call, I- / P-frame (ms)	12.15 / 12.43	13.76 / 13.31	10.71 / 10.66	9.42 / 8.43
Bitrate at QP 26 (kbps)	120.5	287.1	120.5	287.1
Failures / dropped	0 / 0	0 / 0	0 / 0	0 / 0

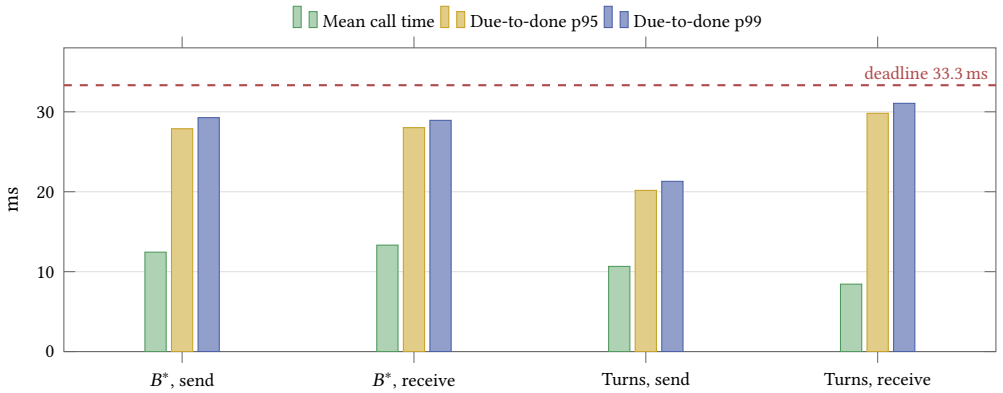


Fig. 4. Real-time behaviour at 30 fps. Every frame meets the deadline, but at the 99th percentile B^* keeps only about 4 ms (12%) of slack.

6.5 E7: Energy

Table 16 gives the power and energy of B^*. While the call ran, the NE drew 1,903 mW, the CPU 415 mW and the GPU nothing measurable; at idle, the NE and GPU drew nothing and the CPU 358 mW. By Eq. (13), the call adds 63.4 mJ per frame pair on the NE and 1.9 mJ on the CPU, 65.3 mJ

in total, which is about 2 W, or 117.6 J per minute of call. The five runs lie within 64.2–66.3 mJ, a spread of ±2%.

Three conclusions follow. First, 97% of the energy of the call is spent on the NE, and the CPU's share is only the cost of coordinating the two threads; the energy is therefore set almost entirely by how long the NE is busy. Second, the GPU is completely idle during the call, so it is spare capacity that DuplexNVC can use. Third, this is the budget against which any change of placement must be judged: moving a network from the NE to the GPU will relieve the NE but will cost GPU energy, and a fair comparison has to report both. The idle subtraction removes the display and the rest of the system, which were held constant by the conditions of Section 3.3, so these numbers are the cost of the codec alone.

Table 16. E7: energy of B^* on AC power from `powermetrics`, idle (120 s) subtracted, median (min–max) of 5 runs; each run uses a steady window of 56 s (1,737 frame pairs).

Processor	**Run (mW)**	**Idle (mW)**	**mJ per pair**	**J per minute**
Neural Engine	1903	0	63.4 (62.5–63.6)	114.2
CPU	415	358	1.9 (1.8–2.9)	3.4
GPU	0	0	0.0	0.0
Total	2318	358	**65.3** (64.2–66.3)	**117.6**

6.6 E5: Decoding Correctness and Drift Across Processors

We took the outgoing stream of the first B^* run, 1,802 frames, and decoded it twice on the NE and once each on the GPU and the CPU. All four decodes completed every frame without an error, and the two NE decodes were identical bit for bit, so decoding on the NE is deterministic: the same stream always gives the same pictures. Together with the absence of failures and drops in all baseline runs, this shows that the baseline is functionally correct.

Decoding on another processor, however, does not reproduce the NE pictures exactly (Table 17, Figure 5). Right after an I-frame, the GPU decode matches the NE decode at 60.6 dB and the CPU decode at 52.6 dB, both far above any visible difference. The difference then grows along the group of pictures: in the last 16 frames before the next I-frame, the mean falls to 47.8 dB (GPU) and 46.1 dB (CPU), and the worst single frames reach 38.5 and 39.5 dB. Over all frames the means are 52.1 and 48.8 dB. The cause is that the decoder's reference for the next frame is its own output: the GPU and CPU compute the networks with slightly different arithmetic from the NE, these small differences are carried forward through the decoder's state, and they add up until the next I-frame resets the state.

We deduce that moving the decoder off the NE, the most obvious way to reduce contention, is possible but not free. On average the difference stays invisible, but the worst frames late in a group of pictures fall to about 38.5 dB, which can be seen. A design that decodes on the GPU therefore needs a bound on drift, for example a shorter I-frame period, and the extra bitrate of that bound must be measured and weighed against the time saved.

6.7 E6: Compression Quality

Against hardware HEVC.. The `mlvc validate` command encoded the three validation clips at QP 16, 24, 32 and 40 and computed the BD-rate of Eq. (18) against the Intel hardware HEVC anchor (Table 18). Full MLVC needs 31.2% fewer bits than the anchor for the same luma PSNR, 73.1% and 72.2% fewer for the two chroma planes, and 41.0% fewer for the weighted PSNR-YUV. Over this

Table 17. E5: Y-PSNR between the NE decode and a GPU or CPU decode of the same stream, by position after the last I-frame (I-frame every 64 frames). Mean / minimum in dB.

Frames after the I-frame	**NE vs. GPU**	**NE vs. CPU**
0	60.58 / 59.58	52.64 / 51.39
1–15	57.63 / 51.90	51.74 / 48.92
16–31	52.86 / 42.73	49.63 / 44.70
32–47	49.93 / 39.52	47.57 / 40.42
48–63	47.80 / 38.46	46.07 / 39.51
All frames	52.13 / 38.46	48.78 / 39.51

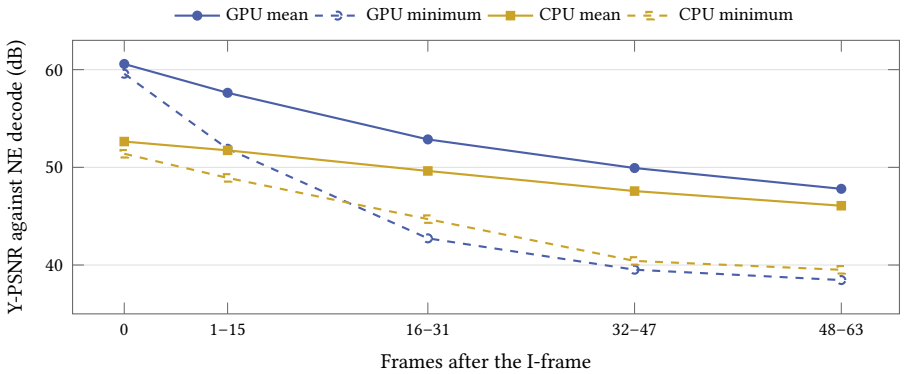


Fig. 5. Drift when the decoder runs on a different processor than the encoder. The difference grows through the group of pictures and resets at each I-frame.

range the anchor spent 82–1,205 kbps and MLVC only 50–507 kbps. The very large chroma savings are typical of learned codecs against a low-power hardware encoder; the luma saving is the more meaningful number, because luma dominates what people see. The baseline codec is therefore clearly more efficient than the hardware codec that laptops use for calls today, which is the reason to use a learned codec at all.

Table 18. E6: BD-rate of full MLVC 360p against Intel hardware HEVC (negative means fewer bits for the same PSNR), on three 3 s VCD s1 clips.

	Y	**U**	**V**	**YUV**	**Anchor kbps**	**MLVC kbps**
BD-rate	**−31.2%**	−73.1%	−72.2%	**−41.0%**	82–1205	50–507

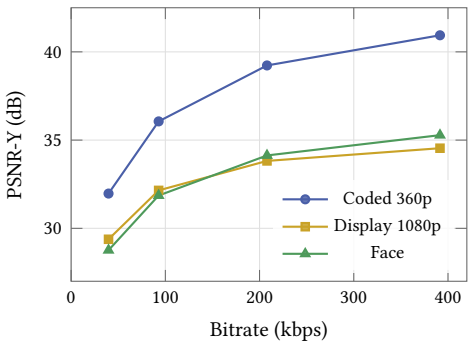
Rate and distortion on the evaluation set. Table 19 and Figure 6 give the means over the 20 evaluation clips. As QP rises from 16 to 40, the bitrate falls from 391.5 to 39.7 kbps, and the coded-scale PSNR-Y falls from 40.94 to 31.97 dB (PSNR-YUV from 42.08 to 34.33 dB). At the display scale of Eq. (16), the same frames score only 34.54 to 29.38 dB, and inside the faces 35.28 to 28.76 dB, while face LPIPS rises from 0.223 to 0.373.

The gap between the coded and the display curves is the main finding of this experiment. Near the baseline QP (between QP 24 and 32, at 120–290 kbps), the coded frame reaches about 36–39 dB, but shown at 1080p it loses another 4–5.5 dB. This loss is caused by the resolution, not by the

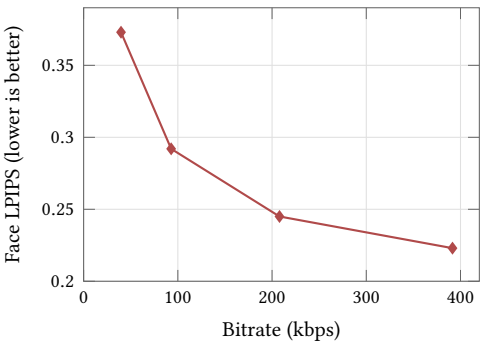
coding, because it remains even at the highest bitrate: at QP 16 the coded frame reaches 40.9 dB but the display only 34.5 dB. The display curve is also much flatter than the coded curve. Halving the bitrate from QP 24 to QP 32 costs 3.2 dB at the coded scale but only 1.7 dB at the display scale, and doubling it from QP 24 to QP 16 gains only 0.7 dB at the display. We deduce that, at 360p, extra bits buy little visible quality, and that the stronger lever is resolution, which costs processing time on the NE that, according to Section 6.4, it does not have to spare. Face PSNR follows the display PSNR closely, and face LPIPS worsens steadily with QP, so faces are not treated differently from the rest of the frame at this point; these two measures will show whether any saving in speed or energy by DuplexNVC is paid for in face quality.

Table 19. E6: full MLVC 360p, mean over the 20 evaluation clips. *Display*: the 360p decode upscaled to 1080p and compared with the 1080p source. *Face*: the display comparison inside the detected face boxes.

QP	kbps	PSNR-Y	PSNR-YUV	Display PSNR-Y	Face PSNR-Y	Face LPIPS
16	391.5	40.94	42.08	34.54	35.28	0.223
24	207.9	39.23	40.62	33.82	34.13	0.245
32	92.9	36.06	37.92	32.15	31.86	0.292
40	39.7	31.97	34.33	29.38	28.76	0.373



(a) PSNR-Y against bitrate.



(b) Face LPIPS against bitrate.

Fig. 6. Rate–distortion of full MLVC 360p on the 20 evaluation clips (QP 40, 32, 24 and 16 from left to right). The gap between the coded and display curves is the loss caused by resolution.

Clip by clip. Figure 7 shows every evaluation clip at QP 24, the standard point closest to the baseline. Apart from two outliers, the clips score 36.9–42.6 dB at the coded scale and lose 2.7–7.6 dB at the display scale. Two clips stand apart: `1e8830` and `4d8ffe` score 25.2 and 25.3 dB at the display scale, about 10 dB below their coded score, and they stay near 23–25 dB at every QP, so their loss does not depend on the bitrate at all. They most likely contain fine texture, such as patterned clothing or a detailed background, that cannot be represented at 360p. Face quality varies more between clips than whole-frame quality (29.8–38.9 dB), because it depends on how large and how detailed each face is. We keep the two outliers in all means, because such content occurs in real calls, and they are exactly the clips on which a higher resolution would help most.

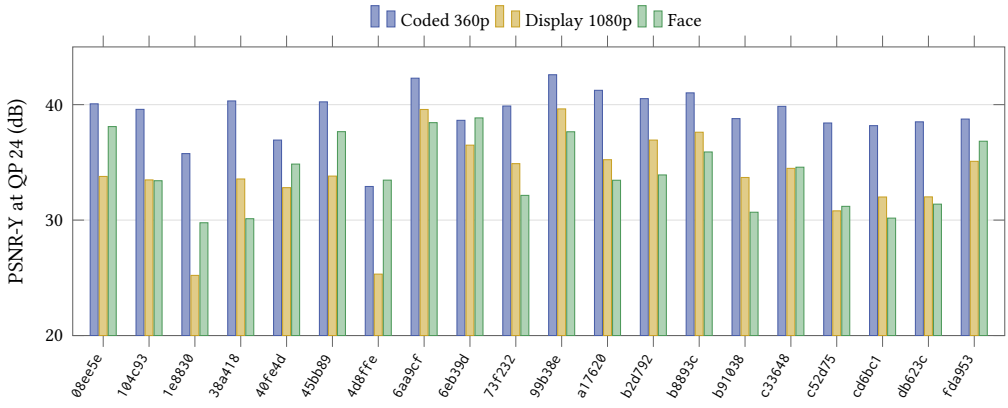


Fig. 7. Quality of each evaluation clip at QP 24, close to the baseline's QP 26. Clips `1e8830` and `4d8ffe` lose about 10 dB at display size.

Timing clips. On the four shipped clips used for the timing streams, the same procedure gave 433.7, 236.9, 104.9 and 43.6 kbps at 40.26, 38.46, 35.33 and 31.48 dB PSNR-Y for QP 16, 24, 32 and 40. These values lie within 1 dB and 15% in bitrate of the evaluation means, so the clips on which we measured speed and energy are representative of the content on which we measured quality.

7 Extending the Baseline: Processors, Resolutions and Placements

The feedback on the first version of this report asked us to place B^* in context before building on it: to show that the networks really run where we ask them to run, to measure the GPU and the CPU as alternatives to the NE, and to measure every model size and resolution of our survey with every assignment of the encoder and the decoder to the three processors. This section reports that extension in three campaigns, X1 to X3. To keep the extension comparable with B^*, it uses the same measurement program, conditions and metrics, and B^* itself is left unchanged (QP 26, an I-frame every 64 frames, both networks on the NE, the two directions due at the same instant). It covers the seven model and resolution pairs of our survey: full MLVC at 360p, 540p, 720p and 1080p, and MLVC-S at 540p, 720p and 1080p. Each pair is run with the encoder and the decoder on the NE and the GPU in all four combinations; the five placements that use the CPU are added only where the encoder and the decoder each take less than the 33.3 ms frame time alone on the CPU, since otherwise they cannot keep up with a 30 fps call.

7.1 X1: Where the Networks Run, and the GPU and CPU Alone

A low NE time could hide operations that Core ML silently runs on the CPU. We therefore wrote a short Swift program that asks Core ML, through its `MLComputePlan` interface, which processor each operation of a compiled model is assigned to and what share of the estimated cost it carries. With the NE requested, the full MLVC networks run 313 of 316 operations on the NE, which carry 100.0% of the cost, and MLVC-S runs 214 of 217 (99.9%); the three remaining operations are tiny input and output conversions on the CPU. With the GPU or the CPU requested, every operation runs on that processor. Every run of X1 and X3 also confirmed, through the library's own report, that the processor used was the one requested.

We then timed each network alone on the NE, the GPU and the CPU, unpaced, in five rounds for full MLVC at 360p and three rounds for MLVC-S at 540p, and paced at 30 fps on the NE and the GPU in three runs of 60 s, with `powermetrics` running. These are 72 runs in total (Table 20).

Table 20. X1: each network alone, median call time in ms, encoder / decoder. MLVC-S uses the shipped perceptual bundle, as in Phase 1.

Model	NE unpaced	GPU unpaced	CPU unpaced	NE paced 30	GPU paced 30
Full MLVC 360p	5.85 / 5.94	13.34 / 13.16	20.00 / 20.68	12.36 / 12.53	15.11 / 14.90
MLVC-S 540p	4.84 / 4.91	10.36 / 10.65	19.73 / 20.95	10.61 / 10.66	12.48 / 12.63

Three results follow. First, unpaced, the GPU is 2.2–2.3 times slower than the NE and the CPU 3.4–4.3 times slower, yet both stay well inside the 33.3 ms frame time, so at these sizes both are real candidates for one of the two networks. Second, paced at 30 fps the gap shrinks to 1.18–1.22, because the NE, like in B^*, takes about twice as long per call when it is paced. Third, a single paced network on the NE takes 12.4 ms, the same as the B^* encoder in full duplex (12.44 ms), which confirms that the doubling of the call time in B^* is caused by pacing and not by the two networks competing. The power samples show the cost of moving work: a network on the GPU drew roughly 3.2–4.6 W on the GPU, against roughly 0.5 W on the NE for the same network, so any placement on the GPU must be judged on energy as well as time.

7.2 X2: New Models and Inputs

We converted full MLVC and MLVC-S with their public PSNR weights (`psnr_v1`) for 540p, 720p and 1080p with `mlvc-tools`, exactly as for B^*, with model inputs padded to a multiple of 16 pixels. We also built one bundle that holds the full model for all four sizes, to test whether one multi-size

bundle runs as fast as single-size bundles. MLVC-S uses PSNR weights in all extension rows, so that full MLVC and MLVC-S are compared under the same training objective; the shipped perceptual bundle is kept only as the link to Phase 1. The placement trace of Section 7.1 was repeated for every new network: at every size the encoder and the decoder run 100.0% of their cost on the NE.

Table 21. X2: converted bundles and the incoming streams at QP 26 (I-frame every 64 frames, encoded on the NE). PSNR is PSNR-Y of the incoming clips.

Setting	Bundle size	Incoming kbps	Incoming PSNR-Y (dB)
Full MLVC 360p (B^*)	–	287.1	34.98
Full MLVC 540p	36 MB	539	36.48
Full MLVC 720p	37 MB	794	37.76
Full MLVC 1080p	41 MB	1438	38.90
MLVC-S 540p	11 MB	561	35.59
MLVC-S 720p	12 MB	847	37.07
MLVC-S 1080p	13 MB	1538	38.43
Full MLVC multi-size (360p–1080p)	41 MB	–	–

The camera and incoming streams use the same four VCD clips as B^*, now scaled from the 1080p releases to each resolution with `ffmpeg`'s Lanczos filter and stored as 600-frame NV12 files. The incoming streams were encoded once at QP 26 and decoded again as a check (Table 21). As expected, the bitrate roughly doubles from 540p to 1080p and quality rises by 2–3 dB; MLVC-S needs 4–7% more bits than full MLVC for 0.5–0.9 dB lower PSNR, which is the price of its smaller networks.

7.3 X3: The 22-Hour Experiment

X3 measures every model and resolution pair with every placement of Section 7 in one script, which ran without interruption for almost 22 hours (1,283 minutes, 584 runs) in quiet mode and can resume after an interruption without repeating finished runs. It has three parts.

(a) *NE sanity and memory* (about 1.9 h): each network of all eight bundles alone on the NE in three rounds, compared with the survey's M3 Pro times, and the peak memory of a duplex run with both networks on the NE and both on the GPU.
(b) *Each network alone on the GPU and the CPU* (about 2.3 h): three rounds for every pair; CPU runs last 20 s with 10 warm-up frames. These times decide automatically which CPU placements run.
(c) *Every placement* (about 17.2 h): for each setting and placement, three unpaced rounds of taking turns and of sharing; five paced duplex runs of 60 s at 30 fps with `powermetrics`; and, if the median late share is 1% or more, three paced runs at 15 fps (deadline 66.7 ms). Each batch starts with a 120 s idle recording for the energy subtraction. B^* itself (360p, NE–NE) was not repeated. The batches ran in the order of the plan: 720p first, then 540p, MLVC-S 720p and 1080p, the 360p placements other than B^*, 1080p, and MLVC-S 540p.

A setting is *feasible at 30 fps* if its median late or dropped share L (Eq. (10)) is below 1% in each direction, *feasible at 15 fps* if it meets that condition only at 15 fps, and *infeasible* otherwise. All metrics are those of Section 2; energy is computed with Eq. (13) against the idle recording of the same batch.

NE sanity against the survey. Table 22 compares the NE times with the M3 Pro times of our survey. Every bundle ran at the expected speed: the M4 Pro needs 0.72–0.76 of the M3 Pro time for full

MLVC and 0.66–0.70 for MLVC-S, so our conversions are as fast as the authors' at every size. The three rounds, spread over about 1.5 hours, agreed to within 0.04 ms for every bundle, which shows that the laptop stayed in the same state throughout. The multi-size bundle ran exactly as fast as the single-size 360p bundle (5.86 and 5.94 ms) but needed 306 instead of 300 MB with both networks on the NE and 687 instead of 487 MB on the GPU, so we keep single-size bundles. Peak memory grows with resolution, from 0.3 GB at 360p to 2.1 GB at 1080p on the NE and 2.4–2.5 GB on the GPU, which is well within the 24 GB of the laptop. The time per frame grows almost in proportion to the number of pixels: from 360p to 1080p the pixels grow ninefold and the full model's time 7.9-fold.

Table 22. X3: NE alone against the survey's M3 Pro times, and peak memory of a 20 s duplex run. Median of three rounds, encoder / decoder in ms; the ratio is that of the sums.

Bundle	M4 Pro, NE	M3 Pro (survey)	M4 / M3	Memory NE–NE	Memory GPU–GPU
Full MLVC 360p (B^*)	5.86 / 5.94	7.72 / 8.14	0.74	300 MB	487 MB
Full MLVC 540p	11.83 / 11.82	15.22 / 16.05	0.76	574 MB	787 MB
Full MLVC 720p	22.23 / 22.17	29.59 / 30.21	0.74	951 MB	1223 MB
Full MLVC 1080p	46.36 / 47.56	64.10 / 65.79	0.72	2092 MB	2449 MB
MLVC-S 540p	4.83 / 4.92	6.58 / 7.27	0.70	527 MB	700 MB
MLVC-S 720p	8.25 / 8.48	11.93 / 13.61	0.66	898 MB	1129 MB
MLVC-S 1080p	18.01 / 18.42	25.32 / 28.90	0.67	1991 MB	2279 MB
Full MLVC multi-size, at 360p	5.86 / 5.94	7.72 / 8.14	0.74	306 MB	687 MB

Each network alone. Table 23 gives the time of each network alone on every processor. At every size the GPU takes 1.9–2.2 times as long as the NE, and the CPU 3.4–4.3 times as long. On the CPU only full MLVC at 360p (19.8 and 20.7 ms) and MLVC-S at 540p (19.7 and 21.3 ms) stay below the 33.3 ms frame time, so only these two settings received the five CPU placements. MLVC-S at 720p missed by a fraction of a millisecond (32.9 and 33.7 ms).

Table 23. X3: each network alone, median call time in ms over three rounds, encoder / decoder, unpaced. CPU runs last 20 s.

Setting	NE	GPU	CPU	GPU / NE	CPU placements run
Full MLVC 360p	5.86 / 5.94	13.33 / 13.18	19.78 / 20.74	2.2	yes
Full MLVC 540p	11.83 / 11.82	25.84 / 26.21	47.4 / 47.7	2.2	no
Full MLVC 720p	22.23 / 22.17	43.22 / 44.07	76.1 / 75.8	2.0	no
Full MLVC 1080p	46.36 / 47.56	93.73 / 97.13	173.0 / 174.5	2.0	no
MLVC-S 540p	4.83 / 4.92	10.34 / 10.68	19.73 / 21.31	2.2	yes
MLVC-S 720p	8.25 / 8.48	16.60 / 17.49	32.9 / 33.7	2.0	no
MLVC-S 1080p	18.01 / 18.42	35.06 / 37.36	71.6 / 74.3	2.0	no

The table of feasible settings. Table 24 is the main result of X3: for every setting and placement, the speedup of sharing over taking turns, the speedup over taking turns on the NE alone, the late share, the slowest frames and the energy, with the resulting label. Of the 38 combinations (37 measured in X3 plus B^*), 23 are feasible at 30 fps, 9 only at 15 fps, and 6 are infeasible. Every run resolved to the processor that was requested. Five findings stand out.

(a) *At 360p and at MLVC-S 540p every placement works.* All nine placements of each meet 30 fps with no late frame (at most 0.1%), with p99 between 15 and 30 ms. Both networks on the NE

is by far the cheapest: 65.3 mJ per frame pair for B^* and 45.0 mJ for MLVC-S 540p, against 125–431 mJ when the GPU or the CPU takes part.

(b) *Both networks on the NE stops working first.* Full MLVC at 540p (6.0% late) and MLVC-S at 720p (3.3% late) fail at 30 fps with both networks on the NE, although their unpaced time per pair (21.7 and 15.1 ms) is well under the frame time. The same settings meet 30 fps when one network is moved to the GPU (NE–GPU and GPU–NE). This is the effect predicted in Section 6.4: paced calls take about twice as long, and the two networks queue on one unit, so the NE alone runs out of time long before its unpaced capacity suggests. Splitting the networks over two processors is the cheapest fix we found.

(c) *Splitting is not free.* For full MLVC 540p the split placements cost 635–672 mJ per pair against 184 mJ on the NE alone, and for MLVC-S 720p 333–383 mJ against 109 mJ, about 3.5 times as much energy, almost all of it on the GPU.

(d) *Both networks on the GPU is never the best choice.* Sharing the GPU gives S of only 1.07–1.22, its time per pair is roughly twice that of NE–NE, and it costs the most energy in every setting. It meets 30 fps only where the networks are small (360p, MLVC-S 540p and 720p).

(e) *Full MLVC at 1080p is infeasible on this laptop*, even at 15 fps, in every placement: each network alone takes 46–97 ms. MLVC-S at 1080p reaches 15 fps in three placements.

In mixed placements S reaches 1.34–1.67, much higher than the 1.08–1.15 of two networks sharing one unit, which confirms that the NE and the GPU do run in parallel; the CPU placements give the highest S (up to 1.67) but also the lowest S_{NE}, because a network on the CPU is slow. When a direction cannot keep up, the decoder, which cannot drop frames, builds a backlog: its p99 then grows to seconds (shown as >1 s), while the encoder drops frames instead.

The best placement of each setting. Taking the cheapest placement at the highest frame rate each setting reaches (shaded in Table 24), the best choice changes with the load.

(a) *Full MLVC at 360p:* NE–NE, that is, B^*. It is 2.6 times cheaper than any other placement (65.3 mJ against 167.5 mJ or more) and keeps the decoder on the NE, so it has no drift. Its only weakness is the margin of the slowest frames (p99 about 29 ms). If more margin is wanted, CPU–NE (encoder on the CPU) gives p99 of 25.3 and 16.3 ms at 167.5 mJ, still with the decoder on the NE.

(b) *Full MLVC at 540p:* GPU–NE. Here NE–NE is *no longer* the best choice, because it misses 30 fps (6.0% of encoded frames late, p99 35.2 ms). Moving the encoder to the GPU removes the contention (0.1% and 0% late, $S = 1.39$) and keeps the decoder on the NE, where it does not drift; it is also slightly cheaper than NE–GPU (635.1 against 672.0 mJ). The price is about 9.7 times the energy of B^*, almost all of it on the GPU, and the encoder's p99 of 31.1 ms leaves little margin.

(c) *Full MLVC at 720p:* NE–NE at 15 fps; no placement reaches 30 fps.

(d) *Full MLVC at 1080p:* none; every placement is infeasible.

(e) *MLVC-S at 540p:* NE–NE, the cheapest combination of the whole table (45.0 mJ per pair) with no late frame and p99 about 27 ms.

(f) *MLVC-S at 720p:* GPU–NE, for the same reason as full MLVC at 540p: NE–NE misses 30 fps (3.3% late), and GPU–NE meets it at 333.4 mJ with the decoder on the NE.

(g) *MLVC-S at 1080p:* NE–NE at 15 fps.

Two rules follow. While both networks fit on the NE in a paced call, they should stay there; once they do not, the encoder, not the decoder, should move to the GPU, because a decoder off the NE drifts (E5) while an encoder off the NE does not.

Table 24. X3: table of feasible settings. Paced values are medians of five 60 s runs at 30 fps, send / receive. S: sharing over taking turns with the same placement (Eq. (5)); S_{NE}: sharing with this placement over taking turns with both networks on the NE (unpaced). Energy with idle subtracted (Eq. (13)). The label is 30 fps if $L < 1\%$ in both directions at 30 fps, 15 fps if this holds only at 15 fps, and infeasible otherwise. Shaded rows mark the best placement of each setting: the cheapest placement at the highest frame rate the setting reaches (Section 7.3).

Setting	Enc–Dec	S	S_{NE}	L (%)	Due-to-done p99 (ms)	mJ per pair	Label
Full MLVC 360p	NE–NE	1.11	1.11	0 / 0	29.3 / 28.9	65.3	30 fps (B^*)
	NE–GPU	1.36	0.82	0 / 0	21.0 / 23.5	212.3	30 fps
	GPU–NE	1.36	0.81	0.1 / 0	23.8 / 21.1	253.4	30 fps
	GPU–GPU	1.19	0.53	0 / 0	29.7 / 28.9	431.6	30 fps
	NE–CPU	1.23	0.55	0 / 0	16.3 / 25.6	170.6	30 fps
	CPU–NE	1.24	0.56	0 / 0	25.3 / 16.3	167.5	30 fps
	GPU–CPU	1.65	0.56	0 / 0	21.8 / 25.9	287.5	30 fps
	CPU–GPU	1.67	0.58	0 / 0	26.0 / 21.7	290.1	30 fps
	CPU–CPU	1.48	0.43	0 / 0	30.0 / 30.1	246.5	30 fps
Full MLVC 540p	NE–NE	1.10	1.10	6.0 / 1.2	35.2 / 33.9	184.2	15 fps
	NE–GPU	1.38	0.85	0 / 0	24.2 / 31.3	672.0	30 fps
	GPU–NE	1.39	0.86	0.1 / 0	31.1 / 25.4	635.1	30 fps
	GPU–GPU	1.13	0.52	100 / 100	80.3 / >1 s	1503.3	15 fps
	CPU	not run: CPU alone 47.4 / 47.7 ms, above 33.3 ms					
Full MLVC 720p	NE–NE	1.08	1.08	100 / 100	74.4 / >1 s	404.8	15 fps
	NE–GPU	1.45	0.97	0 / 100	26.3 / >1 s	1257.9	15 fps
	GPU–NE	1.45	0.99	100 / 0	78.6 / 27.6	1242.4	15 fps
	GPU–GPU	1.07	0.55	100 / 100	117.8 / >1 s	3890.9	**infeasible**
	CPU	not run: CPU alone 76.1 / 75.8 ms, above 33.3 ms					
Full MLVC 1080p	NE–NE	1.08	1.08	100 / 100	120.3 / >1 s	1562.9	**infeasible**
	NE–GPU	1.47	0.94	100 / 100	83.6 / >1 s	4519.5	**infeasible**
	GPU–NE	1.52	0.96	100 / 100	130.2 / >1 s	3779.8	**infeasible**
	GPU–GPU	1.05	0.52	100 / 100	215.8 / >1 s	17620.0	**infeasible**
	CPU	not run: CPU alone 173.0 / 174.5 ms, above 33.3 ms					
MLVC-S 540p	NE–NE	1.15	1.15	0 / 0	26.9 / 26.1	45.0	30 fps
	NE–GPU	1.34	0.82	0 / 0	19.7 / 22.4	125.2	30 fps
	GPU–NE	1.37	0.85	0 / 0	22.3 / 19.8	126.4	30 fps
	GPU–GPU	1.22	0.57	0 / 0	26.2 / 26.0	340.0	30 fps
	NE–CPU	1.18	0.44	0 / 0	15.2 / 26.2	159.2	30 fps
	CPU–NE	1.17	0.47	0 / 0	25.4 / 15.4	150.8	30 fps
	GPU–CPU	1.51	0.46	0 / 0	20.3 / 26.3	276.4	30 fps
	CPU–GPU	1.56	0.49	0 / 0	26.0 / 20.9	288.9	30 fps
	CPU–CPU	1.51	0.35	0 / 0	29.1 / 29.7	234.7	30 fps
MLVC-S 720p	NE–NE	1.12	1.12	3.3 / 0.5	34.5 / 32.7	108.9	15 fps
	NE–GPU	1.35	0.87	0 / 0.1	23.7 / 26.4	383.1	30 fps
	GPU–NE	1.50	0.98	0.1 / 0	25.8 / 24.3	333.4	30 fps
	GPU–GPU	1.14	0.57	0.1 / 0.1	32.6 / 32.4	835.1	30 fps
	CPU	not run: CPU alone 32.9 / 33.7 ms, above 33.3 ms					
MLVC-S 1080p	NE–NE	1.11	1.11	0 / 100	26.8 / 42.3	291.6	15 fps
	NE–GPU	1.41	0.93	0 / 100	23.8 / >1 s	1047.4	15 fps
	GPU–NE	1.46	1.00	100 / 0	69.8 / 25.5	991.0	15 fps
	GPU–GPU	1.09	0.55	100 / 100	99.5 / >1 s	2872.8	**infeasible**
	CPU	not run: CPU alone 71.6 / 74.3 ms, above 33.3 ms					

The 720p question. With the survey's M3 Pro times, full MLVC at 720p needs 59.8 ms per pair when the two networks take turns, which suggested that a speedup of about 1.8 over taking turns on the NE would be needed. On the M4 Pro taking turns on the NE needs 44.6 ms, so the speedup needed falls to 44.6/33.3 ≈ 1.34. Table 25 shows that no placement reaches it: NE–NE gives 1.08, and the split placements only 0.97–0.99, because the GPU half takes twice as long as the NE half. Full MLVC at 720p is therefore feasible only at 15 fps, in three placements, of which NE–NE is the cheapest (405 mJ per pair). MLVC-S at 720p, in contrast, meets 30 fps in three placements: NE–GPU, GPU–NE and GPU–GPU; the cheapest of these, GPU–NE, uses 333 mJ per pair. For 720p at 30 fps on this laptop, the smaller model is required, and one of its networks must leave the NE.

Table 25. X3: 720p by placement. Unpaced T_{shared} and speedups; paced late share at 30 fps (send / receive); paced energy per frame pair by processor (mJ, idle subtracted).

Model	**Enc–Dec**	T_{shared} **(ms)**	S	S_{NE}	L **(%)**	**NE**	**GPU**	**CPU**	**Total**	**Label**
Full MLVC	NE–NE	41.33	1.08	1.08	100 / 100	397.8	0.0	6.6	404.8	15 fps
	NE–GPU	45.86	1.45	0.97	0 / 100	179.6	1055.1	23.0	1257.9	15 fps
	GPU–NE	45.11	1.45	0.99	100 / 0	240.1	977.0	25.1	1242.4	15 fps
	GPU–GPU	80.92	1.07	0.55	100 / 100	0.0	3830.4	59.8	3890.9	infeasible
MLVC-S	NE–NE	15.08	1.12	1.12	3.3 / 0.5	107.0	0.0	1.9	108.9	15 fps
	NE–GPU	19.47	1.35	0.87	0 / 0.1	35.5	342.4	5.3	383.1	30 fps
	GPU–NE	17.17	1.50	0.98	0.1 / 0	37.8	290.4	5.1	333.4	30 fps
	GPU–GPU	29.84	1.14	0.57	0.1 / 0.1	0.0	815.2	19.8	835.1	30 fps
Speedup needed		Full MLVC: 44.6/33.3 = 1.34; MLVC-S: 16.9/33.3 = 0.51 (taking turns on the NE, M4 Pro)								

Settings run at 15 fps. The 15 combinations that missed 30 fps were measured again at 15 fps (Table 26). Nine of them meet 15 fps with no late frame, including every placement of full MLVC at 540p and 720p that failed at 30 fps and three of MLVC-S at 1080p. The six infeasible ones are all four placements of full MLVC at 1080p and GPU–GPU at 720p (full MLVC) and 1080p (MLVC-S).

Stability. The chip's thermal state was logged once per second for the whole run. The repeated rounds show no drift: the NE times of the three sanity rounds differ by at most 0.04 ms, and the five paced runs of each placement took the same time to complete, so neither heat nor background activity affected the results.

Table 26. X3: combinations that failed at 30 fps, measured again at 15 fps (deadline 66.7 ms), median late share, send / receive.

Setting	Enc–Dec	L at 30 fps (%)	L at 15 fps (%)	Label
Full MLVC 540p	NE–NE	6.0 / 1.2	0 / 0	15 fps
Full MLVC 540p	GPU–GPU	100 / 100	0 / 0	15 fps
Full MLVC 720p	NE–NE	100 / 100	0.1 / 0	15 fps
Full MLVC 720p	NE–GPU	0 / 100	0 / 0	15 fps
Full MLVC 720p	GPU–NE	100 / 0	0 / 0	15 fps
Full MLVC 720p	GPU–GPU	100 / 100	100 / 100	infeasible
Full MLVC 1080p	NE–NE	100 / 100	100 / 100	infeasible
Full MLVC 1080p	NE–GPU	100 / 100	0 / 100	infeasible
Full MLVC 1080p	GPU–NE	100 / 100	100 / 0	infeasible
Full MLVC 1080p	GPU–GPU	100 / 100	100 / 100	infeasible
MLVC-S 720p	NE–NE	3.3 / 0.5	0 / 0	15 fps
MLVC-S 1080p	NE–NE	0 / 100	0 / 0	15 fps
MLVC-S 1080p	NE–GPU	0 / 100	0 / 0	15 fps
MLVC-S 1080p	GPU–NE	100 / 0	0 / 0	15 fps
MLVC-S 1080p	GPU–GPU	100 / 100	74.1 / 58.5	infeasible

8 Discussion

The baseline answers the questions it was built for, and Table 27 collects the answers. Full MLVC runs a 360p duplex call in real time on an M4 Pro, with no late frame but only about 4 ms of slack for the slowest frames. Switching between the two models costs nothing, but running them at the same time on the NE gives a speedup of only 1.11 out of a possible 1.99, because each network slows down by a factor of about 1.8. The call uses 65 mJ per frame pair, 97% of it on the NE, while the GPU is idle. Decoding is correct and deterministic on the NE, but decoding on the GPU or CPU drifts within each group of pictures, down to 38.5 dB in the worst frame. The codec needs 31% fewer bits than hardware HEVC, and the largest loss of visible quality comes from the 360p resolution rather than from the coding. Finally, the Phase 1 results for MLVC-S hold to within 4%.

Taken together, these results support the motivation of DuplexNVC and make it more precise. The problem is real: a duplex call on one Neural Engine is limited by contention, since the two networks run largely one after the other, the slowest frames come close to the deadline, and nearly all the energy goes to the one busy unit while the GPU is idle. Running both networks in parallel on the NE is not the way forward, because the measurements rule it out. Three directions remain:

(a) *placement*, running one of the two networks on the GPU, provided that the drift found in E5 is kept within bounds and that the GPU's energy is counted;
(b) *scheduling*, making the two directions due at different moments so that they do not compete at the same instant, which targets the start delay that makes up a large part of the slowest frames; and
(c) *resolution*, spending the time gained by (a) and (b) on a larger picture, since E6 shows that resolution matters most for what the viewer sees.

The extension confirms this reading at every size. Wherever both networks fit on the NE they should stay there, because it is several times cheaper than any other placement; but the NE alone is the first placement to fail as resolution or model size grows, well before its unpaced capacity is

Table 27. Summary of the core baseline B^*.

Question	Answer
Does full MLVC run a 360p duplex call in real time on an M4 Pro?	Yes: 30 fps both ways, 0% late, p99 about 29 of 33.3 ms.
Does switching between the two models cost anything?	No: about 0.01 ms.
How much does running both at once on the NE help?	Little: $S = 1.11$ against an ideal of 1.99; $\kappa \approx 1.8$.
Energy?	65 mJ per frame pair (about 2 W), 97% on the NE; GPU idle.
Is decoding correct?	Yes, and deterministic on the NE; GPU or CPU decoding drifts within a group of pictures (worst 38.5 dB).
Compression against HEVC?	BD-rate −31% (Y), −41% (YUV).
Largest loss of visible quality?	The 360p-to-1080p upscale (4–6 dB), more than coding.
Do the Phase 1 MLVC-S results hold?	Yes, within 4%; $S = 1.15$ is now measured.
Does any operation fall back to the CPU?	No: 100% of the cost of every network runs on the requested processor.
GPU and CPU as alternatives?	GPU 2.2–2.3× slower (1.2× paced) at several times the power; CPU 3.4–4.3× slower.
Which settings are feasible at 30 fps?	23 of 38: all of 360p and MLVC-S 540p; full MLVC 540p and MLVC-S 720p only with one network off the NE; full MLVC 720p only at 15 fps; full MLVC 1080p never.

used up, and moving one network to the GPU recovers 30 fps for full MLVC at 540p and MLVC-S at 720p at a large energy cost. A controller therefore has a real choice to make, between resolution, model size, placement and energy, and the table of feasible settings (Table 24) is the space from which DuplexNVC will choose.

8.1 Balancing Latency, Quality and Energy

The extension turns the baseline into a trade-off between three quantities: the delay a frame adds to the call, the quality the viewer receives, and the energy the laptop spends. Table 28 places the cheapest feasible placement of each setting side by side with the quality of its incoming stream at the same QP.

Three points stand out.

(a) *Quality costs frame rate before it costs latency.* Every setting that meets 30 fps keeps its slowest frames within 24–31 ms, so latency at 30 fps hardly depends on the setting. What the heavier settings lose is the frame rate: full MLVC at 720p and MLVC-S at 1080p reach their higher quality only at 15 fps, which doubles the frame interval to 66.7 ms and makes motion less smooth, and full MLVC at 1080p cannot run at all. For a call, the real limit is therefore not how late a frame is, but how many frames per second the chip can deliver at a given quality.

(b) *The steps up in quality are expensive.* Going from 360p to 540p or 720p raises the PSNR of the incoming stream by 0.6–2.8 dB, and E6 showed that resolution, not coding, causes most of the loss that a viewer sees at 360p. But the settings that deliver this at 30 fps beyond MLVC-S 540p, full MLVC at 540p and MLVC-S at 720p, need one network on the GPU, and cost 333–635 mJ per frame pair, five to ten times B^*.

Table 28. Latency, quality and energy of the cheapest placement of each setting. Latency is the paced due-to-done p99 (send / receive) at the frame rate in the label; energy is per frame pair with idle subtracted, measured at 30 fps; bitrate and PSNR-Y are those of the incoming stream at QP 26 (Table 21), measured at the coded resolution.

Setting	Placement	Label	p99 (ms)	mJ per pair	kbps	PSNR-Y (dB)
Full MLVC 360p (B^*)	NE–NE	30 fps	29.3 / 28.9	65.3	287	34.98
MLVC-S 540p	NE–NE	30 fps	26.9 / 26.1	45.0	561	35.59
Full MLVC 540p	GPU–NE	30 fps	31.1 / 25.4	635.1	539	36.48
MLVC-S 720p	GPU–NE	30 fps	25.8 / 24.3	333.4	847	37.07
Full MLVC 720p	NE–NE	15 fps only	–	404.8	794	37.76
MLVC-S 1080p	NE–NE	15 fps only	–	291.6	1538	38.43
Full MLVC 1080p	any	infeasible	–	–	1438	38.90

(c) *MLVC-S at 540p is a better operating point than B^*.* With both networks on the NE it is faster (p99 about 27 ms against 29 ms), uses less energy (45.0 against 65.3 mJ per pair) and gives a larger picture with a higher PSNR (35.6 against 35.0 dB), at about twice the bitrate. The smaller model at a higher resolution beats the larger model at a lower one on every measure except bitrate.

These PSNR values are measured at each setting's own resolution, so they understate the gain a viewer sees from a larger picture; the display-scale comparison of all settings at equal bitrate is step N1 of the plan, and it is what will tell DuplexNVC how much quality each extra millijoule buys. The trade-off itself is already clear: on this laptop, quality is bought with frame rate, energy, or bitrate, and the controller must choose among them according to the content of the call and the energy state of the device.

8.2 Limitations

The results are subject to the following limitations:

(a) The full model uses the public `psnr_v1` weights, converted by us; production weights may differ in speed and quality.
(b) The 1080p reference is the compressed MP4 release of VCD, so display and face PSNR are lower bounds on the true fidelity.
(c) `powermetrics` reports estimates from the chip's internal power models, not metered power, and all energy runs were on AC power on one device; battery power and Low Power Mode are not yet measured.
(d) Each run lasts 60 s, so behaviour over a long call, when the chip heats up and may slow down, is untested.
(e) Time Machine could not be paused; the small spread between runs suggests it had no effect, but it was not excluded.
(f) Only talking-head content (VCD `s1`) was evaluated, and only 40 of its clips were downloaded; scenes with several people or small faces are not yet covered. The data planned for them is described in Section 3.6.1.
(g) The lower NE clock between paced frames is a likely explanation of the doubled call time, not a measurement.
(h) We could not check the `powermetrics` estimates against an external power meter (an inline USB-C meter between the charger and the laptop, or a wall-socket meter), because we did

not have access to such hardware. If it is provided, the check is planned as part of the energy step of Section 9.

(i) The public checkpoints used for all models are trained on the OpenVid dataset, whereas the MLVC paper used models trained on Vimeo data, so speeds match the paper but quality may differ from it.

(j) The extension covers timing and energy only: quality at the new settings, correctness of every encoder and decoder pairing, and operation on battery are not yet measured (Section 9).

9 Plan for the Next Phase

The plan has two parts. The first (Section 9.1) completes and extends the baseline, so that every setting DuplexNVC may choose from is fully measured. The second (Section 9.2) is the project itself: building DuplexNVC on top of this baseline and showing what it gains over B^*.

9.1 Completing and Extending the Baseline

This report establishes the baseline and its extension: the core baseline B^* (Section 6), the placement trace and the processors alone (X1), the new models and inputs (X2), and the timing and energy of every setting and placement (X3). The other steps of our plan could not be completed by the deadline. Each needs several hours with the laptop in quiet mode, during which it cannot be used for anything else or connected to a network; X3 alone took almost 22 hours. We therefore present them here as the first part of the plan. The baseline steps not yet implemented, in the order in which we will carry them out, are:

(a) *quality at every setting* (N1): rate, distortion, display and face quality of all seven settings, and the Pareto front of energy against perceptual quality, which completes the table of feasible settings;

(b) *correctness of every encoder and decoder pairing* (N2) at 360p and 720p, which shows whether moving a network to another processor keeps the codec correct;

(c) *the program extensions* (N3) needed by the next two steps;

(d) *energy beyond AC power* (N4): long calls, battery, Low Power Mode, and the check against an external power meter if the hardware is provided;

(e) *the cost of scene analysis* (N5) inside the call, which must stay below about 2 ms per frame pair;

(f) *the controller data* (N6): full VCD, HEVC Class E and the Pexels multi-person clips of Section 3.6.1, with faces labelled and grouped by size;

(g) *an on-device HEVC anchor* (N7) measured on the same laptop.

Table 29 lists each step with its method, the condition under which it is complete and a rough estimate of the laptop time it needs; Figure 8 shows their order.

External power meter. The energy results of this report come from `powermetrics`, which reports the estimates of the chip's internal power models. The plan foresaw checking them against an external meter, an inline USB-C power meter between the charger and the laptop or a wall-socket meter. We could not do this check because we did not have access to such hardware. If the hardware is provided, the check will be added to step N4: the same B^* runs and idle recordings will be measured with the meter and with `powermetrics` at the same time, and the difference, after subtracting idle in both, will be reported. Until then, all energy values are internal estimates, as stated in Section 8.2.

9.2 Building DuplexNVC on the Baseline

The second part of the plan builds DuplexNVC itself: a controller that runs both directions of a call across the NE, the GPU and the CPU and chooses MLVC's settings from the content of the frame

Table 29. Plan for the next phase. Times are rough estimates of laptop time; "quiet" means the step needs quiet mode (Table 4).

#	Step	Method	Complete when	Time
N1	Quality at every setting	The E6 method on the 20 evaluation clips at QP 16, 24, 32 and 40 for all seven settings: coded, display and face PSNR, LPIPS; comparison at equal bitrate by interpolating along the four-QP rate–distortion curves, since `libmlvc` sets a QP per frame and has no target-bitrate mode; Pareto plot of energy against LPIPS from X3.	Every row of Table 24 has a quality value; Pareto front drawn.	4–6 h (not quiet)
N2	Correctness of every pairing	Encode on NE, GPU or CPU and decode on each (nine pairs) at 360p and 720p, 20 clips, four QPs: failures, drift by position after the I-frame, change in BD-rate, determinism. Optionally I-frame period 32 and 16 for a GPU decoder.	3×3 tables for 360p and 720p; Change in BD-rate known (target: below 1 point).	6–8 h
N3	Program extensions	Separate encoder and decoder bundles; power source in the per-run state file; a hook that runs scene analysis every N frames.	One paced B^* run within 1% of this report.	1 h quiet
N4	Energy beyond AC power	B^* and the best 720p setting: 30 minutes on AC; battery above 80%; battery below 30%; Low Power Mode; a 60-minute drain. Idle per state; battery voltage and current at 1 Hz; `pmset -g batt`. Check of `powermetrics` with an external meter (below).	Every scenario has a row; 30-minute trace.	5–6 h quiet plus charging
N5	Cost of scene analysis	Apple Vision face detection inside B^*: off; every 15 frames; pinned to the CPU; CPU with tracking every 30 frames. Detection recall at 320×180 against 1080p by face size.	Each variant has a row; target ≤ 2 ms per frame pair and no late frame.	1–2 h quiet
N6	Controller data	Full VCD (160 clips), HEVC Class E (1, 2 and 4 people) and eight Pexels clips with 2–7 people entering and leaving (Section 3.6.1); retimed to 30 fps and scaled to 360p, 540p and 720p; face boxes on every 10th source frame; face-size groups (<32, 32–63, 64–127, ≥128 pixels at 360p); calibration and evaluation split; B^* face quality per group.	Data converted, labelled and split; Table 6 complete.	150 GB download and conversion
N7	On-device HEVC anchor	Apple VideoToolbox HEVC in low-latency mode at MLVC's bitrates on the evaluation clips and Class E at 360p and 720p; BD-rate on four metrics; duplex energy over 5×60 s including the media engine and memory, also for B^*.	BD-rate and energy measured over the same parts of the chip.	4–6 h quiet

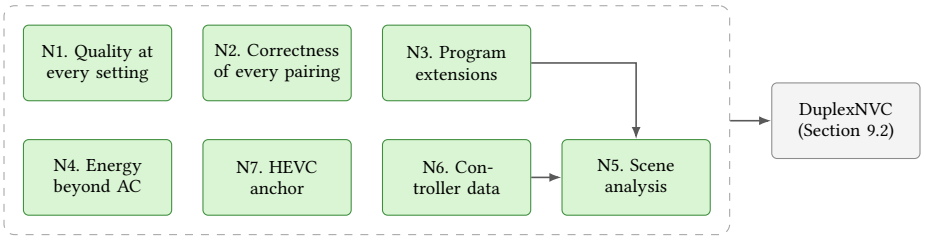


Fig. 8. Order of the next steps. N1, N2, N3, N6 and N7 can start at once; N4 needs the power-source record of N3, and N5 needs the scene-analysis hook of N3 and the controller data; their results feed the DuplexNVC part of the plan.

and the energy state of the laptop. The baseline results above define what it must do and against what it will be judged. Its components follow directly from the measurements:

(a) *Placement.* Choose, per call, where the encoder and the decoder run. X3 shows that keeping both on the NE is cheapest wherever it meets the deadline, and that moving one network to the GPU is what makes full MLVC at 540p and MLVC-S at 720p feasible, at about 3.5 times the energy. The controller uses the table of feasible settings (Table 24) to pick the cheapest placement that meets 30 fps.
(b) *Scheduling.* Offset the two directions in time, by a quarter or a half of a frame period, so that they do not queue on the same unit at the same instant; Section 6.4 shows that this waiting makes up much of the slowest frames, and X3 shows that it is what makes NE–NE fail first.
(c) *Resolution and model.* Spend the time and energy saved on what the viewer sees: E6 shows that resolution, not coding, limits displayed quality at 360p, so the controller raises resolution or switches between full MLVC and MLVC-S when the budget allows.
(d) *Adaptation to content and energy.* Use the scene analysis measured in N5 (number and size of faces) and the battery and Low Power Mode state measured in N4 to move between settings during a call, within the cost of about 2 ms per frame pair.
(e) *Drift control.* Where a decoder runs on a different processor from the NE, bound the drift found in E5 and N2, for example with a shorter I-frame period, and count the extra bitrate this costs.
(f) *Evaluation against B^*.* Measure every DuplexNVC configuration with exactly the metrics, conditions and protocol of this report, so that each gain in S, p99, energy or quality, or its absence, is measured against the same unchanged baseline B^*.

Later steps of the project extend this to a real call between two laptops over a network and to calls with more than two participants, where the load on each chip grows further.

10 Conclusion

We defined and measured the core baseline B^* for DuplexNVC: full MLVC at 360p with both networks on the Apple Neural Engine, running a 30 fps two-way call on an M4 Pro. We built a C++ measurement program on `libmlvc` that records every frame, checked it against Phase 1 and against the library's own tool, and measured the baseline under fixed conditions with Wi-Fi off, the screen at a fixed brightness and the laptop on AC power. The baseline meets real time with no late frame, but with only about 4 ms of slack for the slowest frames. It uses 65 mJ per frame pair, almost all on the NE, and needs 31% fewer bits than hardware HEVC. The key measurement is that sharing the NE between the encoder and the decoder gives a speedup of only 1.11 out of a possible 1.99: the NE runs the two networks largely one after the other. Together with the finding that resolution, not coding, limits the quality the viewer sees, this defines the opening for DuplexNVC. Relieving the NE, by placing one network elsewhere and by scheduling the two directions, can buy both time and resolution, as long as the drift between processors is kept within bounds. The extension of Section 7 began this work: every network runs on the processor it is given, the GPU is a real but costly alternative to the NE, and a single 22-hour experiment now measures every model size, resolution and placement of our survey, whose results complete the table of feasible settings. The measurements also define the balance DuplexNVC must strike: at 30 fps latency stays within 24–31 ms for every feasible setting, so higher quality is paid for in frame rate, energy or bitrate rather than delay, and MLVC-S at 540p on the NE already beats B^* on speed, energy and picture size (Section 8.1). For full MLVC the extension also sets the limits of the baseline itself. At 360p, B^* is the best placement on every measure, but 360p is too low a resolution: E6 shows that upscaling it to the screen costs 4–6 dB, more than the coding does. At 540p, both networks on the NE are

no longer the best choice, because they miss 30 fps; the best placement becomes the encoder on the GPU and the decoder on the NE, which meets 30 fps without drift but at about ten times the energy of B^*. Above 540p, full MLVC does not reach 30 fps on this laptop at all, and the smaller MLVC-S at 540p with both networks on the NE is the most efficient setting we measured. This report is deliberately limited to the baseline. Every remaining step needs several hours with the laptop in quiet mode, so they are given as the plan on which DuplexNVC will be built and against which it will be judged.

References

[1] Shams Al-Ani, Warda Ul Hasan, and Mohammed Alawadhi. 2026. Full-Duplex Neural Video Coding on Laptop SoCs: A Survey and Proposed Research Direction. Survey report, American University of Sharjah.

[2] Gisle Bjøntegaard. 2001. Calculation of Average PSNR Differences between RD-curves. ITU-T SG16/Q6 VCEG, Document VCEG-M33.

[3] Zhaoyang Jia, Bin Li, Jiahao Li, Wenxuan Xie, Linfeng Qi, Houqiang Li, and Yan Lu. 2025. Towards Practical Real-Time Neural Video Compression. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 12543–12552. doi:10.1109/CVPR52734.2025.01170

[4] Jiahao Li, Bin Li, and Yan Lu. 2021. Deep Contextual Video Compression. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[5] Babak Naderi, Ross Cutler, Nabakumar Singh Khongbantabam, Yasaman Hosseinkashi, Henrik Turbell, Albert Sadovnikov, and Quan Zou. 2024. VCD: A Video Conferencing Dataset for Video Compression. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*.

[6] Tanel Pärnamaa, Martin Lumiste, Ardi Loot, Evgenii Indenbom, Andrei Znobishchev, and Ando Saabas. 2026. MLVC: Multi-platform Learned Video Codec for Real-World Deployment. In *Proceedings of the European Conference on Computer Vision (ECCV)*. To appear. arXiv:2606.28027.

[7] Frank Bossen. 2013. Common Test Conditions and Software Reference Configurations. JCT-VC, Document JCTVC-L1100.

[8] Pexels. 2026. Free Stock Videos. https://www.pexels.com/videos/. Used under the Pexels license.

[9] Gary J. Sullivan, Jens-Rainer Ohm, Woo-Jin Han, and Thomas Wiegand. 2012. Overview of the High Efficiency Video Coding (HEVC) Standard. *IEEE Transactions on Circuits and Systems for Video Technology* 22, 12 (2012), 1649–1668.

[10] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. 2018. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 586–595.